

Performance analysis of social network models in agent based systems

Martí Gimeno Ortí

Universitat Oberta de Catalunya. 2025-2026

Table of Contents

1 Introduction	7
1.1 Problem description	7
1.2 State of the art	7
1.3 Objectives	8
1.4 Methodology	9
1.5 Planning	9
1.6 Ethical, social and environmental impact	10
2 Model descriptions	11
2.1 Misinformation diffusion model	11
2.1.1 Epidemiological foundations: The baseline SBFC framework	12
2.1.2 Architectural enhancements: Heterogeneous agent classes	15
2.1.3 Experimental input parameters	16
2.1.4 Validation methodology	17
2.2 Socio-epistemic knowledge spread model	19
2.2.1 Historical and empirical validation context	19
2.2.2 Architecture and internal computational logic	21
2.2.3 Agent interaction mechanisms and movement dynamics	23
2.2.4 Experimental input parameters and validation	26
3 Model implementations	27
3.1 Misinformation diffusion model	27
3.1.1 Parallelizing the model	27
3.1.2 Validating the Repast HPC implementation	29
3.2 Socio-epistemic knowledge spread model	33
3.2.1 Parallelizing the model	33
3.2.2 Modifications to the original model	34
3.2.3 Validating the Repast HPC implementation	35
4 Performance Analysis	39
4.1 Methodology	39
4.2 Misinformation diffusion model results	42
4.3 Socio-epistemic knowledge spread model results	48
Conclusions	52
Annex I: Source Code and Reproducibility	55
Bibliography	56

List of Figures

Figure 1	States and transitions of the SBFC model	13
Figure 2	Forces that drive the movement of an agent through the epistemic landscape	26
Figure 3	Repast HPC memory management applied to the SBFC model	29
Figure 4	Validation of the Repast HPC implementation for parameter set 1 . .	31
Figure 5	Validation of the Repast HPC implementation for parameter set 2 . .	31
Figure 6	Validation of the Repast HPC implementation for parameter set 3 . .	31
Figure 7	Initial epistemic landscape used for the validation process	35
Figure 8	Validation of the Repast HPC implementation for parameter set 1 . .	36
Figure 9	Validation of the Repast HPC implementation for parameter set 2 . .	36
Figure 10	Validation of the Repast HPC implementation for parameter set 3 . .	36
Figure 11	Trajectory of agents in the epistemic space over time in the Repast HPC implementation with parameter set 2	38
Figure 12	Degree distribution for, left to right, Holme-Kim, Watts-Strogatz and Barabási-Albert.	42
Figure 13	Execution time breakdown per partitioning strategy.	43
Figure 14	MPI message size by message direction per partition strategy.	43
Figure 15	Total execution time for each network type, positioned by network average degree and average clustering coefficient. Interpolated results on the right.	44
Figure 16	Execution time breakdown per number of total MPI ranks for Holme-Kim (left) and Barabási-Albert (right) networks.	45
Figure 17	Execution time breakdown per number of agents in absolute (left) and relative (right) time.	45
Figure 18	Execution time breakdown for different combinations of ranks and agents for Holme-Kim (left) and Barabási-Albert (right) networks. . .	46
Figure 19	Execution time breakdown per extra computational load in absolute (left) and relative (right) time.	47
Figure 20	Execution time breakdown per extra MPI message size in absolute (left) and relative (right) time.	47

Figure 21	Execution time breakdown per partitioning strategy.	49
Figure 22	MPI message size by message direction per partition strategy.	49
Figure 23	Total execution time for each network type, positioned by network average degree and average clustering coefficient. Interpolated results on the right.	50
Figure 24	Execution time breakdown per number of total MPI ranks.	51
Figure 25	MPI Synchronization time per number of total MPI ranks.	51

List of Tables

Table 1	Gantt diagram for task planning	10
Table 2	Agent type characteristics of the heterogeneous SBFC model	16
Table 3	Input parameters of the heterogeneous SBFC model	17
Table 4	Node source methodology for the "General Relativity Social Space" network	21
Table 5	Input parameters of the socio-epistemic knowledge spread model	26
Table 6	Parameter sweeps for validating the Repast HPC implementation of the misinformation diffusion model	30
Table 7	Scientist agent internal attributes	33
Table 8	Parameter sweeps for validating the Repast HPC implementation of the socio-epistemic model	36
Table 9	Description of the timers used for performance analysis and execution time breakdown.	40
Table 10	Default configuration for the performance analysis of the misinformation diffusion model.	43
Table 11	Default configuration for the performance analysis of the socio-epistemic knowledge spread model	48

Chapter 1

Introduction

1.1 Problem description

Agent Based Modelling is a computational method used to simulate the actions and interactions of autonomous agents (individuals, companies, cells) and study their effects on the system as a whole. Its goal is to understand complex phenomena by watching it emerge from the individual interactions between a large number of agents. The complex group behavior is not modelled nor programmed directly, but it naturally occurs, as in an ant colony.

High Performance Computing (HPC) is the practice of aggregating computing power in a way that delivers much higher performance than a single workstation could deliver. It relies on the concept of parallel processing in order to break down a single job with considerable computational work into multiple smaller jobs that can be processed at the same time. All HPC systems are composed of nodes, which perform the calculations, and interconnections, which allow for communication between nodes.

Given the parallel nature of agent based systems, it's logical to combine the two disciplines to unlock the ability to simulate millions of agents in reasonable time. But, can one quantify the real performance gain of implementing an agent based model in a high performance computing platform? Which are the performance bottlenecks of the HPC platforms that accelerate ABMS? Is there any way to alleviate those? This Master thesis work will answer the mentioned questions by studying the performance of Agent Based systems implemented in High Performance Computing platforms.

1.2 State of the art

Over the past two decades, several frameworks for implementing agent-based modeling and simulation (ABMS) on high-performance computing (HPC) systems have been proposed and applied to real-world case studies. Among the most prominent examples are FLAME [1], FLAME GPU [2], RepastHPC [3], [4] and DMASON [5]. These frameworks are designed to allow domain specialists to build large-scale simulations without directly dealing with the low-level complexities of parallelism. Despite this shared motivation, they adopt different design choices: agents and environments may be defined using diverse languages and formalisms, inter-agent interactions can rely on distinct communication strategies, and different support for features such as load balancing. These architectural and implementation choices influence scalability and

efficiency, leading to substantially different performance outcomes depending on the framework being chosen and the simulation model characteristics.

Benchmarks on the performance discrepancies between different HPC frameworks and ABMS characteristics have been performed by [6] and [7], shedding light on the problem of which HPC framework to select depending on the simulation model. This work focuses in the RepastHPC framework to implement the parallelized versions of the agent based models. It is selected for its C++ based architecture designed specifically for TOP500 class supercomputers, utilizing MPI (Message Passing Interface) for efficient cross-process communication and spatial decomposition.

Regarding availability of open-source real-world agent-based models, the Computational Model Library provided by the CoMSES Net [8], [9] has been the primary database for exploring and publishing ABMS since its publication more than a decade ago. This platform acts as a common sharing ground of peer-reviewed, reproducible models that follow the standard ODD protocol, providing a reliable foundation for the performance analysis.

1.3 Objectives

The main objective of the work is to evaluate the performance and identify the bottlenecks of two existing complex agent based models implemented in the Repast HPC framework. To achieve that, the following intermediate objectives are presented:

1. **Models research and selection:** Selecting two representative social network agent based models, that could be implemented in an HPC framework to support millions of agents.
2. **Detailed analysis for the selected models:** The exhaustive analysis will include descriptions on the model internal logic, the agent relationship and communication, the complexity of the agent and environment logic and the inputs and outputs of the model.
3. **Implementing the selected ABMS in an HPC framework:** Description of the Repast HPC framework fundamentals and how each model has to be implemented according to Repast HPC concepts like the Scheduler, the Context and MPI communication. Validation and verification of the programs to ensure that the model has been implemented properly.
4. **Simulation execution and analysis performance:** Benchmark executions with parameter sweeping and scalability tests to measure performance metrics like execution time, resource usage, and parallel efficiency.
5. **Bottleneck identification:** The main part of the analysis will be centered around identifying, quantifying and describing the nature of the inefficiencies and bottlenecks of each model.

1.4 Methodology

To achieve the mentioned objectives in the previous section, the following methodology will be applied

- **ABMS literature review:** Use CoMSES Net model repository and academic databases to analyse and select two social networks agent based models. In order to validate the model selection, the following properties will be analyzed:
 - *Model domain:* Description of the model representation, inputs, outputs and purpose.
 - *Agent relationship:* Number of agent kinds and the relationship between them, description of their communication mechanism, content and criteria.
 - *Model complexity:* Evaluation of the underlying logic for agents and environment.
- **Implementing the ABMS in an HPC framework:** Select the appropriate framework using the benchmarks designed for ABMS performance evaluation for HPC in [7] and [10].
- **Define testing environment and performance metrics:** Each model will be evaluated under multiple scenarios and parameters to identify their effects on the chosen performance metrics.

1.5 Planning

Follows an enumeration and description of all the tasks needed to achieve the work's objectives:

1. ABMS research and selection

- 1.1. Introduction to CoMSES Net's Computational Model Library.
- 1.2. Preselection of most interesting ABMS found with an initial superficial analysis.
- 1.3. Final selection of 2 ABMS that fit the criteria defined previously.

2. Models in-depth analysis

- 2.1. Description of the model domain, inputs, outputs and purpose.
- 2.2. Description of the model agents behaviour, relationship and parameters.
- 2.3. Identification of the parameter sweeping for performance evaluation.

3. HPC implementation

- 3.1. Introduction to HPC frameworks and their unique characteristics.
- 3.2. Introduction to RepastHPC.
- 3.3. Implementation of models with RepastHPC.

4. Execution and validation

- 4.1. Programs validation to ensure that the models have been implemented properly.
- 4.2. Executions with parameter sweeping and scalability tests to measure performance metrics: execution time, resource usage, parallel efficiency...

5. Performance analysis

- 5.1. Analysis of performance metrics results.
- 5.2. Identification of bottleneck and possible causes.

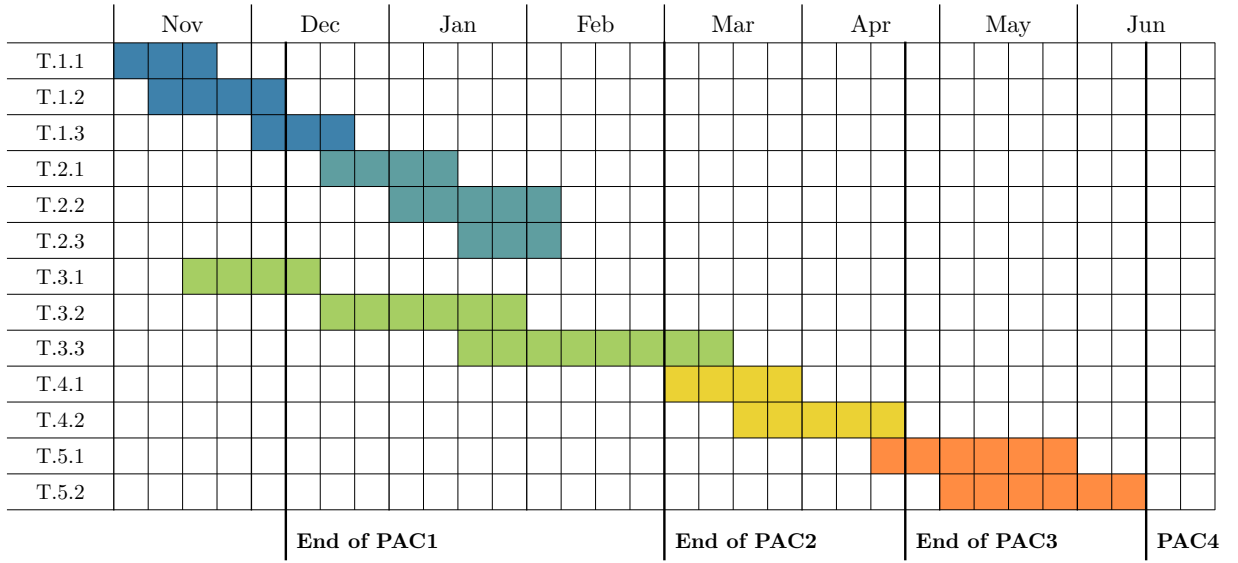


Table 1: Gantt diagram for task planning

1.6 Ethical, social and environmental impact

This work contributes to the advancement of computational social science by enabling larger and more accurate simulations of complex systems. By identifying performance bottlenecks in economic and social network models, this research facilitates the creation of digital twins that can be used by governments to predict economic crises or the spread of misinformation with higher fidelity.

Ethically, the work promotes transparency in algorithmic decision-making by optimizing open-source tools like RepastHPC and relying on open-access model libraries like CoMSES, democratizing access to powerful simulation capabilities. Also, the code produced by this work will be published in an open-source repository under the MIT license.

Environmentally, the focus on performance optimization and bottleneck identification directly relates to energy efficiency. By reducing the computational resources required to run massive simulations, the carbon footprint of large-scale research projects can be minimized.

Chapter 2

Model descriptions

The evaluation of High Performance Computing frameworks for Agent-Based Modelling and Simulation requires the selection of baseline models that are robust, computationally significant, and relevant to contemporary science. Following the methodology and objectives outlined in the introductory chapter, the models selected for this performance analysis operate within the domain of computational social science, specifically focusing on the diffusion of information in a social network. This chapter provides an exhaustive analysis of the models internal mechanisms, mathematical foundations, agent interactions, and the specific variables that will govern the subsequent HPC implementation and benchmarking phases.

2.1 Misinformation diffusion model

The first selected model, archived in the COMSES Net Computational Model Library under version 1.0.0, is titled “*Controlling the Misinformation Diffusion in Social Media by the Effect of Different Classes of Agents*” [11], [12] and describes the spread of misinformation and the corresponding countermeasures in digital social networks.

This model represents an architectural extension of the foundational Susceptible-Believer-FactChecker (SBFC) epidemic model originally proposed in [13], [14]. While the original SBFC framework established the mathematical model for simulating the competitive diffusion of a viral hoax and its debunking information within a homogeneous population, the selected extended model introduces significant topological and behavioral complexities that mirror real-world dynamics. By categorizing the network nodes into distinct, heterogeneous agent types: Scholars, Influencers, Normal Agents, and Bots, the model accurately captures the asymmetric influence and dynamic belief structures present in real-world platforms like Facebook [15].

From a computational perspective, the introduction of heterogeneous agent classes and targeted intervention strategies significantly increases the computational load required for each agent evaluation cycle. Furthermore, simulating this environment over a large-scale, highly connected graph introduces intricate memory access patterns and inter-agent communication dependencies that suppose a challenge to parallelize efficiently. These characteristics are precisely what expose the performance bottlenecks, parallelization inefficiencies, and synchronization overheads inherent in distributed-memory platforms such as Repast HPC.

2.1.1 Epidemiological foundations: The baseline SBFC framework

The Susceptible-Believer-FactChecker model conceptualizes the spread of misinformation not as a simple, unidirectional contagion, but as a complex, simultaneous competitive process between a virus (the hoax) and its cure (the debunking information). By modeling hoaxes analogously to biological viruses, the framework borrows heavily from traditional compartmental epidemic models such as the Susceptible-Infected-Recovered (SIR) and Susceptible-Infected-Susceptible (SIS) paradigms. However, it splits the “Infected” compartment into two distinct and adversarial sub-compartments, creating a more dynamic state space.

I. Agent State Space and Network Topology

The simulation environment is mathematically represented as a network graph $G = (V, E)$, where V represents the set of vertices (the individual agents or users) and E represents the set of edges (the social connections or follower relationships between them). At any discrete time step t , each agent is characterized by a specific behavioral state: This state is rigorously defined by a triple of binary indicators:

$$\forall i \in V \quad s_{i(t)} = [s_i^B(t), s_i^F(t), s_i^S(t)] = \begin{cases} [1, 0, 0] \\ [0, 1, 0] \\ [0, 0, 1] \end{cases} \quad (1)$$

The state vector is constrained such that it can only assume one of three mutually exclusive configurations, corresponding to the agent’s current cognitive stance regarding the circulating information:

- **Believer (B)**: Represented as $s_i^B(t) = 1$. In this state, the agent i has been exposed to the hoax, believes it to be true, and actively participates in its propagation across the network.
- **Fact-Checker (FC)**: Represented as $s_i^F(t) = 1$. In this state, the agent i has either independently verified the news or been exposed to debunking information. The agent recognizes the information as a hoax, adopts the factual narrative, and actively propagates the truth to its neighbours.
- **Susceptible (S)**: Represented as $s_i^S(t) = 1$. This is a neutral state. The agent i is either entirely ignorant of the circulating narrative, or they have forgotten a previously held belief (either belief in the hoax or belief in the facts).

II. Stochastic Transition Dynamics

The temporal evolution of the system is governed by a stochastic, discrete-time Markov process. The probability that an individual agent i transitions into or remains in a specific state at time $t + 1$ is defined by a probability mass function:

$$\begin{aligned}
p_i(t+1) &= [p_i^B(t), p_i^F(t), p_i^S(t)] = \\
&= \begin{cases} p_i^B(t+1) = f_i s_i^S(t) + (1 - p_{\text{forget}} - p_{\text{verify}}) s_i^B(t) \\ p_i^F(t+1) = g_i s_i^S(t) + p_{\text{verify}} s_i^B(t) + (1 - p_{\text{forget}}) s_i^F(t) \\ p_i^S(t+1) = p_{\text{forget}} (s_i^B(t) + s_i^F(t)) + (1 - f_i - g_i) s_i^S(t) \end{cases} \quad (2)
\end{aligned}$$

where p_{forget} , p_{verify} are constant probabilities and f_i , g_i are the spreading functions that provide the network effect. Figure 1 shows the three states and their primary sociocognitive phenomena that drive these probability distributions and subsequent transitions: Forgetting (p_{forget}), Verifying (p_{verify}) and Spreading (f_i , g_i).

The actual, deterministic state of the agent at the next sequential time step is then determined by a random realization of this distribution:

$$s_i(t+1) = \text{MultiRealize}[p_i(t+1)]. \quad (3)$$

III. The Spreading Mechanism

Two continuous spreading functions, $f_i(t)$ and $g_i(t)$, calculate the exact probability of a Susceptible agent i adopting the hoax or adopting the debunking information, respectively. These functions evaluate the ratio of Believers and Fact-Checkers within the agent's immediate network neighborhood, modulated by global system parameters:

$$f_i(t) = \beta \frac{n_i^B(t) \cdot (1 + \alpha)}{n_i^B(t) \cdot (1 + \alpha) + n_i^F(t) \cdot (1 - \alpha)} \quad (4)$$

$$g_i(t) = \beta \frac{n_i^F(t) \cdot (1 - \alpha)}{n_i^B(t) \cdot (1 + \alpha) + n_i^F(t) \cdot (1 - \alpha)} \quad (5)$$

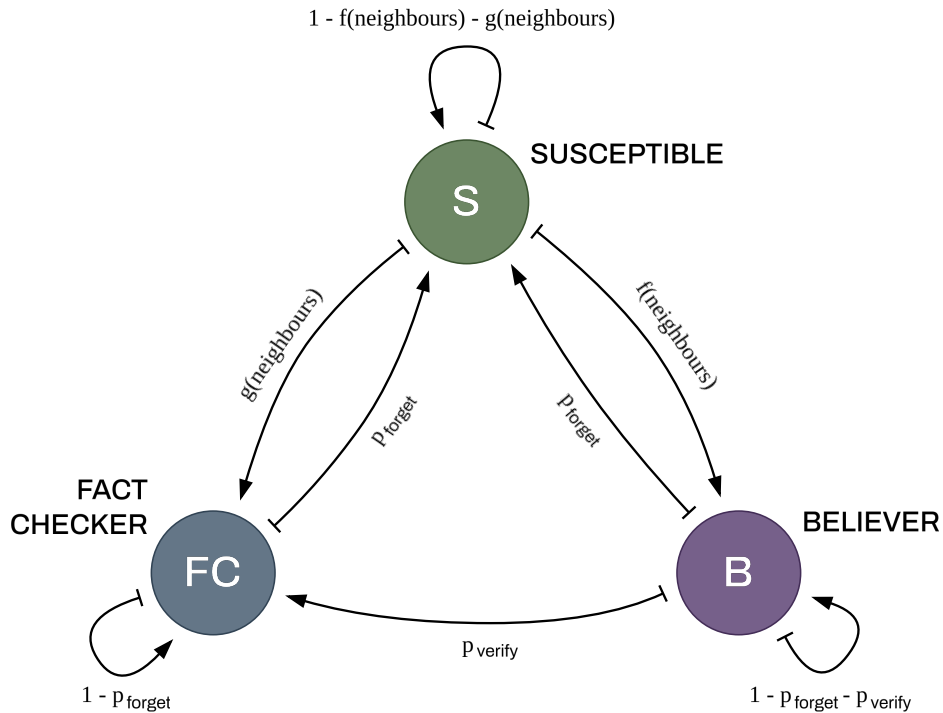


Figure 1: States and transitions of the SBFC model

In these formulations, $n_i^B(t)$ and $n_i^F(t)$ represent the absolute, integer count of agent i 's direct neighbours who are currently in the Believer and Fact-Checker states at time t . The parameter $\beta \in [0, 1]$ represents the base spreading rate, defining the overall transmissibility or virality of information across the network, independent of its truthfulness. The parameter $\alpha \in [0, 1]$ is a constant parameter for the credibility of the hoax. It is critical to note that $f_i(t) + g_i(t) = b\eta$, which aligns the total aggregate infection rate with traditional SIS epidemic models, ensuring mathematical consistency.

From a high-performance computing perspective, the evaluation of $n_i^B(t)$ and $n_i^F(t)$ represents one of the most significant computational burdens in the simulation. For an agent to calculate its spreading functions, the processor must fetch the state data of every single vertex connected to agent i . In dense, scale-free networks, a single agent may possess thousands of edges, requiring massive, non-contiguous memory access operations that frequently result in degraded performance.

IV. Results

The original authors of the SBFC model conducted extensive mean-field approximations to understand the infinite-time limit behavior of the system, assuming a homogeneous network where all vertices have the same degree k . Their mathematical analysis demonstrated that the density of susceptible individuals at equilibrium (S_∞) depends solely on the spreading rate β and the forgetting probability p_{forget} , calculated as:

$$S_\infty = \frac{p_{\text{forget}}}{\beta + p_{\text{forget}}} \quad (6)$$

The core analytical breakthrough of the baseline model, however, was the derivation of a hard threshold for the verification probability that guarantees the complete eradication of the hoax from the network, achieving a state where $B_\infty = 0$. By setting the equilibrium equation for Believers to zero and solving for p_{verify} , the sufficient condition for hoax eradication is expressed as:

$$p_{\text{verify}} \geq \frac{2\alpha}{1 - \alpha} \cdot p_{\text{forget}} \quad (7)$$

This critical equation establishes that the minimum required fact-checking effort to eradicate a hoax is not linear. Instead, it scales non-linearly with the hoax's inherent credibility α and proportionally with the network's overall forgetting rate. If a hoax is highly credible (α approaches 1), the required verification effort approaches infinity. While this elegant mathematical threshold holds true for homogeneous, mean-field approximations, the dynamics alter significantly, and become vastly more difficult to predict, when network topologies exhibit real-world scale-free properties and when agent behaviors become heterogeneous. This limitation in the baseline model necessitates the transition to the extended model selected for this thesis.

2.1.2 Architectural enhancements: Heterogeneous agent classes

The fundamental flaw in utilizing the baseline SBFC model to analyze real-world social networks is its assumption of a homogeneous population. In the baseline model, every node in the graph exhibits the exact same propensity to verify information (p_{verify}), the exact same rate of memory decay (p_{forget}), and equal susceptibility to peer influence. Real digital social ecosystems, however, are characterized by vast disparities in user influence, media literacy, engagement algorithms, and automated bot activity.

The model selected for this HPC performance analysis addresses these limitations by introducing four distinct, behaviorally heterogeneous classes of agents. This extension pushes the model closer to reality and simultaneously increases the algorithmic complexity of the simulation, making it an excellent candidate for benchmarking distributed computing systems.

This enhancement transforms the localized neighbor counting mechanism. The transition equations that calculate $n_i^B(t)$ and $n_i^F(t)$ must now aggregate the influences of neighbours separated by their respective classes, requiring the simulation to query not only the state of each neighbor, but also its inherent class type during every iteration:

$$n_i^B(t) = n_{\text{SC}_i}^B(t) + n_{\text{I}_i}^B(t) + n_{\text{N}_i}^B(t) + n_{\text{BOT}_i}^B(t) \quad (8)$$

$$n_i^F(t) = n_{\text{SC}_i}^F(t) + n_{\text{I}_i}^F(t) + n_{\text{N}_i}^F(t) + n_{\text{BOT}_i}^F(t) \quad (9)$$

where the subscripts SC, I, N, and BOT denote Scholars, Influencers, Normal agents, and Bots, respectively. Follows a description of each agent type characteristics, summarized in Table 2:

- **Normal agents:** Normal agents constitute the vast, overwhelming majority of the network graph. They represent the standard, everyday users of a social media platform who consume and share information without specialized agendas or elevated media literacy. In the simulation, the Normal class operates strictly according to the baseline parameters originally defined in the homogeneous SBFC model. Specifically, they exhibit a standard, relatively high forgetting probability ($p_{\text{forget}} = 0.1$) and a low baseline verification probability ($p_{\text{verify}} = 0.05$). Because they represent the numerical bulk of the population, the global macroscopic state of the simulation, that is whether the hoax ultimately becomes an endemic fixture of the network or is successfully eradicated, heavily depends on the shifting beliefs of this specific class. Their behavior provides the underlying substrate through which the other, more specialized classes operate.
- **Scholars:** The Scholar class represents a minority subset of users characterized by high media literacy, a deeper understanding of factual information, and a proactive approach to debunking falsehoods. Behaviorally, Scholars exhibit a substantially higher probability of independently verifying incoming news, with experimental configurations testing $p_{\text{verify-scholar}} \in [0.05, 0.3]$. Furthermore, because their beliefs are rooted in verified evidence rather than fleeting trends, they exhibit a significantly lower probability of forgetting their verified stance, with $p_{\text{forget-scholar}} \in [0.02, 0.1]$.

A critical topological constraint distinguishes the Scholar class from the rest of the population: they are not randomly or uniformly distributed across the network graph. Instead, they are densely concentrated within specific, highly interconnected structural clusters, simulating real-world academic, scientific, or professional knowledge silos.

- **Influencers:** Influencers model human users who possess disproportionate social reach and exceptionally high network centrality. Topologically, they are rigidly defined as the top 1% of nodes sorted by out-degree connectivity. Because they are connected to a massive number of follower neighbours, any state change in an Influencer rapidly cascades through the network, creating massive localized ripples in the $f_i(t)$ and $g_i(t)$ functions of thousands of normal agents. Behaviorally, Influencers are modeled to hold onto their public beliefs longer than Normal agents to maintain their brand identity, resulting in a lower p_{forget} . They also possess a variable, but generally higher, verification probability than normal users, simulating the resources they might have to evaluate information before broadcasting it to their large audiences.
- **Bots:** The Bot class introduces fully automated, algorithmic, and static agents into the social ecosystem. Bots represent the top 10% of nodes in terms of degree connectivity, explicitly excluding the already designated human Influencers. Crucially, Bots do not participate in the cognitive Markov transitions of the SBFC framework. They possess a zero probability of forgetting ($p_{\text{forget}} = 0$) and a zero probability of verification ($p_{\text{verify}} = 0$). Once initialized at $t = 0$, their state is permanently immutable. Bots are further subdivided into two adversarial factions: Believer Bots and Fact-Checker Bots.

2.1.3 Experimental input parameters

Evaluating the performance of this model within an HPC framework requires a rigorous, systematic parameter sweep. The experimental space must deliberately push the simulation across varied phases of execution: from states where the hoax dies out rapidly (resulting in low computational interaction and early equilibrium) to states of intense, enduring, network-wide competition between Believer and Fact-Checker cascades (resulting in high computational interaction and maximum resource utilization).

The defined input variables and their specific tested values, derived from the original literature, are detailed in Table 3.

	Topology	p_{verify}	p_{forget}
Normal	Uniformly distributed	0.05	0.1
Scholar	Densely concentrated	$\in [0.05, 0.3]$	$\in [0.02, 0.1]$
Influencer	Nodes with top 1% of edges	$\in [0.05, 0.2]$	$\in [0.02, 0.1]$
Bot	Nodes with top 10% of edges	0.0	0.0

Table 2: Agent type characteristics of the heterogeneous SBFC model

Param	Description	Values
α	The inherent believability factor of the hoax. Higher values artificially increase the mathematical weight of Believer neighbors in the spreading functions.	$\{ 0.3, 0.8 \}$
β	The base transmission intensity determining how quickly beliefs spread from neighbors to a Susceptible agent, independent of truth.	$\{ 0.5, 0.75 \}$
B_0	The percentage of the total agent population initialized in the Believer state at $t = 0$, setting the initial infection seed.	$\{ 10\%, 40\% \}$
$p_{\text{verify}}^{\text{SC}}$	The specific probability that an agent in the Scholar class will independently verify the news.	$\{ 0.05, 0.1, 0.2, 0.3 \}$
$p_{\text{forget}}^{\text{SC}}$	The specific probability that a Scholar agent will forget their belief and revert to Susceptible.	$\{ 0.02, 0.05, 0.1 \}$
$p_{\text{verify}}^{\text{I}}$	The specific probability that an agent in the Influencer class will independently verify the news.	$\{ 0.05, 0.1, 0.2 \}$
$p_{\text{forget}}^{\text{I}}$	The specific probability that an Influencer will forget their belief and revert to Susceptible.	$\{ 0.02, 0.05, 0.1 \}$
BOT_B	The percentage of top-degree nodes (excluding Influencers) algorithmically converted into static Believer Bots.	$\{ 0\%, 1\%, 2\%, 5\% \}$
BOT_F	The percentage of top-degree nodes (excluding Influencers) algorithmically converted into static Fact-Checker Bots.	$\{ 0\%, 1\%, 2\%, 5\% \}$
Ticks	The temporal duration of the simulation. Each tick represents one hour. 168 ticks represent a 7-days lifecycle.	168

Table 3: Input parameters of the heterogeneous SBFC model

2.1.4 Validation methodology

To verify that the newly developed C++ Repast HPC implementation correctly captures the intricate dynamics of the original model, a rigorous validation protocol must be established. Because the underlying transition functions are highly stochastic, direct 1:1 validation of a single run is mathematically impossible; instead, the validation framework must rely on demonstrating statistical equivalence over large sample sizes.

The primary metrics for validation will be the mean densities of the three agent states B_∞ , FC_∞ , and S_∞ , at the conclusion of the 168-tick simulation cycle. The baseline experiments in the original literature generated 4 replicates for each of the 13,824 settings. Analysis of these replicates revealed a standard deviation of less than 5% relative to the total agent population, indicating good macroscopic model stability despite individual micro-level stochasticity.

The Repast HPC model will be executed using identical input parameter matrices and identical network topology. The resulting macroscopic curves must statistically match the baselines within a strict confidence interval. For example, an interesting

dynamic to replicate would be the non-linear growth of Believers when hoax credibility is maximized ($\alpha = 0.8$), versus the total systemic dominance of Fact-Checkers when hoax credibility is minimized ($\alpha = 0.3$). Additionally, the localized, systemic effects of the heterogeneous classes, such as the specific 8-9% reduction in the global Believer population when the Scholar class is properly configured and educated, must be demonstrably reproducible in the parallelized environment.

Only once this statistical parity is achieved, the performance analysis phase can be started. The benchmark suite will then systematically scale the agent population and the allocated MPI processes, recording total execution time, memory overhead, parallel efficiency, and MPI communication latency to fully expose, analyze, and document the architectural bottlenecks of distributing complex social network models across High Performance Computing environments.

2.2 Socio-epistemic knowledge spread model

The second selected model is the socio-epistemic framework titled “*An Opinion Dynamics of Science? Agent-Based Modeling of Knowledge Spread*”. Authored by Bernardo Buarque and published via the CoMSES Net Computational Model Library under version 1.0.0 [16], this model was developed under the guidance of the ModelSEN project, an initiative dedicated to modeling historical knowledge processes and the evolution of science [17].

The model operates as a socio-epistemic simulation engine, conceptually based in the literature of opinion dynamics and the sociology of scientific knowledge. It is inherently multi-layered, simulating both the tangible social networks of researchers, represented as a graph, and the abstract epistemic spaces they navigate, represented as clusters in a two-dimensional spatial lattice. The fundamental purpose of the model is to identify and isolate the socio-structural mechanisms that drive the creation, diffusion, and eventual consensus of mental models within a bounded scientific community. By simulating the interplay between scientists, institutional proximity, and conceptual coherence, the model aims to answer how marginal ideas transition into mainstream paradigms.

For the specific purposes of this Master’s Thesis performance analysis, this socio-epistemic model presents an ideal candidate for HPC implementation. It exhibits complexities that include dynamic network topologies that evolve over time, continuous spatial density evaluations, heterogeneous agent decision-making logic, and computationally intensive initialization phases. By translating the authors original NetLogo-based implementation into the C++ Message Passing Interface (MPI) architecture of Repast HPC, the subsequent phases of this research will accurately quantify parallelization efficiency, measure cross-node communication overhead, and isolate resource bottlenecks.

The following analysis systematically decomposes the historical validation parameters, the internal computational logic, and the specific algorithms that must be optimized for distributed execution.

2.2.1 Historical and empirical validation context

To ensure the simulation generates sociologically valid outputs, the model is based in the historical phenomenon known as the *Renaissance of General Relativity* [18], [19]. This paradigm shift in theoretical physics, which occurred between 1925 and 1970, provides the foundational validation parameters and the empirical edge-list data utilized when the simulation operates in its historically driven mode. Understanding the exact contours of this historical dataset is critical, as any successful HPC implementation must not only execute efficiently but also accurately replicate the emergent macro-phenomena observed in the historical record.

I. The Dynamics of the General Relativity Renaissance

The status of the General Theory of Relativity underwent a radical and structurally profound transformation across a four-decade timespan. The historical consensus divides this evolution into two highly distinct phases, which the socio-epistemic model attempts to replicate via emergent agent behavior and spatial clustering.

During the initial phase, commonly referred to as the *Low-Water Mark* (spanning from the mid-1920s to the mid-1950s), the theory maintained a highly marginal status within the broader physics community, often relegated to the domain of abstract mathematical curiosity. Research during this period was characterized by extreme fragmentation. It was pursued primarily by isolated individuals or small, disconnected academic groups operating under divergent epistemic agendas, such as unified field theory, the quantization of Einstein’s equations, and theoretical cosmology. Co-authorship and cross-institutional collaboration were markedly sparse, reflecting a deeply divided epistemic landscape.

The subsequent phase, termed the *Renaissance* (spanning from the mid-1950s to 1970), witnessed the theory’s dramatic return to the scientific mainstream, eventually solidifying as a foundational pillar of modern physics. Historically, this transition was characterized by the rapid establishment of a highly uniform research field focused heavily on the theory’s tangible physical and astrophysical predictions. Traditional historical narratives often attribute this renaissance to sudden external astrophysical discoveries, most notably the observational discovery of quasars in 1963.

However, extensive multilayer network analyses conducted by researchers associated with the ModelSEN project have empirically disproven this external-trigger hypothesis [18], [20]. The historical data demonstrates that a fundamental structural shift in the underlying social collaboration network of physicists occurred between the late 1950s and early 1960s, significantly prior to these external astrophysical discoveries. The core mathematical signature of this shift was the sudden formation of a “giant component” within the social network graph, indicating a rapid, large-scale integration of previously isolated research clusters into a unified socio-epistemic community. The fundamental validation target of the ABM is to simulate the precise localized agent interaction rules that organically precipitate the emergence of this giant component without relying on external exogenous shocks.

II. Empirical Network Parameters

When the simulation is initialized using empirical data rather than procedural generation, it relies on highly specific network parameters extracted through analysis. The construction of this validation dataset required to capture the full picture of scientific interaction, acknowledging that simple co-authorship records are insufficient to map the true flow of knowledge. The node population of the model, termed the *General Relativity Social Space*, comprises a total of 971 unique authors acting as individual agents, described in Table 4. This population was aggregated through meticulous historical data curation and bibliometric extraction [21].

Node Source	Identification Criteria	Count
Web of Science (WoS) Extraction	Authors publishing at least two distinct items within the defined general relativity publication space between 1925 and 1970.	770
Manual Historical Supplementation	Authors identified via manual historical research (e.g., conference proceedings, review articles, and the Mathematics Genealogy Project up to 1971) to correct for missing or misindexed online records.	201
Total network nodes		971

Table 4: Node source methodology for the "General Relativity Social Space" network

The underlying *General Relativity Publication Space* used to define these nodes and calculate their initial epistemic positions consists of 8.296 indexed articles. This corpus was assembled through a multi-tiered search strategy. It began with an initial citation space anchored by Albert Einstein’s foundational papers from 1915 to 1955, expanding outward to include two subsequent generations of citing literature. This citation mapping was then heavily augmented by multi-lingual keyword searches incorporating terms such as “quantum gravity”, “unified field”, and “cosmology” across English, German, French, and Italian databases.

In the computational implementation of the model, agent activity is strictly temporally constrained to match this empirical reality. An agent node only exists in the active simulation space during its documented duration of activity, spanning strictly from the exact year of its first relevant publication to the year of its final recorded contribution. This empirical rigor provides a stable, deterministic benchmark against which the generative, stochastic outputs of the simulation’s movement logic can be statistically validated across parameter sweeps.

2.2.2 Architecture and internal computational logic

I. General model description

The architecture of the NetLogo implementation relies on a highly coupled, dual-layer environment representing a conceptual synthesis of continuous spatial simulation and discrete network theory. This dual topology requires agents to manage coordinates in a Cartesian space while simultaneously maintaining references to a graph structure. This structural duality forms the core challenge for subsequent HPC translation.

The model comprises several distinct computational entities that interact across a simulated timespan. The primary environment is composed of a continuous two-dimensional space with a discrete grid embedded in it, representing the landscape of possible mental models. The moving agents represent the individual scientists navigating this

epistemic landscape. The relationships between these scientists are instantiated as explicit network links, forming the social topology of the simulation.

The state variables defining these entities dictate the limits of their interaction. Each grid node belongs to a mental model, represented with different colours. The agents hold a birth date and a boolean flag indicating temporary assignment to academic conferences. This birth variable functions not only as a timestamp for temporal activation but mathematically serves as a proxy for age and epistemic rigidity, permanently altering the agent's velocity across the simulation space. Agents possess the capability to accurately perceive the logical validity of their immediate spatial environment, recognising specific grid colours as coherent topics while identifying black grid nodes as logically inconsistent conceptual spaces.

The interactions driving the simulation are heterogeneous, preventing uniform convergence. Agent interactions span from purely topological network transversals, where an agent consults its social graph regardless of geographic distance, to entirely spatial gradient ascents, where an agent ignores its social ties and simply moves toward the highest local concentration of peers. This interplay between social influence and spatial popularity generates the complex dynamics required to replicate the historical case study.

The primary observation metric used to evaluate the terminal state of the simulation is defined as the *Epistemic Share*. The system continuously tracks the global distribution of the active agent population across the distinct topical clusters established during initialization. By outputting a dynamic time-series plot of the proportional density of agents residing within each defined conceptual region, one can quantitatively measure the degree of conceptual polarization versus consensus achieved by the end of the simulation lifecycle.

II. The epistemic layer and K-means initialization

The cognitive space of the scientific community is modeled as a continuous two-dimensional regular lattice. Each discrete spatial coordinate within this lattice is bound to a mental model. The spatial arrangement of the environment is inherently semantic; the Euclidean distance between any two points directly correlates to their conceptual similarity. For example, neighboring lattice nodes might represent two nearly identical theoretical physics models that differ solely by the inclusion or exclusion of a single causal variable or mathematical constant. Agents navigating this space are effectively engaging in epistemic exploration, gradually transitioning their beliefs from one conceptual paradigm to another.

The setup phase of this epistemic layer is computationally intensive, as it procedurally generates the semantic landscape utilizing a k-means clustering algorithm. The procedure initializes by instantiating 10,000 points, symbolizing the foundational empirical corpus of scientific literature. These points are not distributed uniformly across the grid. Instead, they are aggregated into six distinct regions representing broad scientific disciplines or topical clusters. The underlying code utilizes a normal distribution function

combined with a predefined standard deviation parameter to dictate the variance and spatial spread of these points around randomized initial coordinates.

To formalize these abstract regions into concrete geometric zones, the model invokes its clustering routine. Eight centroid points are instantiated and placed at the exact same coordinates as eight of the points representing literature entities, randomly chosen. Then, each of the 10.000 points calculates its Euclidean distance to all active centroids and assigns itself to the nearest one, updating its internal color state to match the assigned centroid. Finally, each centroid recalculates its own coordinate position to match the exact mathematical mean of the X and Y coordinates of all data points currently assigned to it. The 10.000 temporary data points are then discarded.

III. The social layer and temporal graph dynamics

Superimposed directly over the epistemic lattice is a topological graph representing the social and institutional ties between the scientific agents. The initialization of this network layer is highly configurable, offering two distinct operational modes governed by the global input parameters.

In the procedural generation mode, the system utilizes a preferential attachment algorithm to simulate an idealized, scale-free academic network. This generates a highly centralized structure where a small minority of nodes possess a vast majority of connections, effectively simulating the disproportionate influence of elite researchers.

Conversely, in the empirical data mode, the system invokes file input operations to instantiate the network via historical data integration. The model parses an external dataset to place pre-configured agents with specific birth dates and absolute initial coordinates onto the lattice. As the simulation clock advances, a continuous background process reads a temporal edge list, dynamically creating and destroying topological links between agents precisely when the simulation tick matches the historical timestamp of the real-world interaction. This dual capacity allows the model to function both as an abstract theoretical sandbox and as a rigid historical reconstructive tool.

2.2.3 Agent interaction mechanisms and movement dynamics

The core execution loop of the model simulates the passage of time over 100 discrete ticks. During each tick, agents execute a series of decision-making algorithms that determine their trajectory across the epistemic lattice. These movements are strictly governed by cognitive constraints and a highly specific set of attraction forces, all mathematically modulated by a universal velocity equation.

IV. Cognitive constraints: absorptive capacity, cognitive coherence and the movement speed

The system implements theoretical concepts from the sociology of science to create rigid boundaries on agent behavior:

- The first constraint is Cognitive Coherence: The model states that an agent cannot logically adopt a mental model that is fundamentally inconsistent or scientifically

invalid. In the spatial context of the simulation, grid nodes colored black represent these void spaces or incoherent theories. If an agent's calculated trajectory causes it to land on a black lattice node, the movement is immediately nullified, and the agent is forced to revert to its original coordinate position. This ensures agents only navigate through validated epistemic territory.

- The second constraint is Absorptive Capacity: The theoretical premise asserts that scientists are incapable of making radical, instantaneous leaps in scientific thought. An agent must evaluate the cognitive Euclidean distance between its current location and its intended focal point. The movement algorithm is aborted entirely if this distance exceeds the universally defined distance threshold, effectively bounding the scope of conceptual change achievable in a single interaction cycle.

When an agent successfully targets a valid focal point within its absorptive capacity, the velocity of its traversal, defined as the step size taken per tick, is not linear. Instead, the model mathematically dictates the movement speed through a rigorously bounded logistic decay function:

$$\text{speed} = \frac{1}{1 + e^{-(\text{birth} \times \text{distance})}} \quad (10)$$

This specific mathematical formulation is the core of the agent behavioral dynamics, yielding sociological implications:

- The **birth** variable represents the simulation tick at which the agent entered the system. The speed formula dictates that agents with lower birth values (which conceptually equates to older, more established, and tenured scientists) move slower across the epistemic lattice. This simulates academic rigidity and the historically observed decreasing likelihood of senior scientists radically altering their established theoretical paradigms.
- The **distance** variable further modulates the speed's sigmoid curve. This exponential relationship ensures that agents decelerate asymptotically as they approach their conceptual target, mimicking the gradual nature of scientific consensus building, where the final minor theoretical alignments take substantially longer than initial broad agreements.

The five forces of epistemic attraction

During the execution phase, agents select their focal points based on five distinct, configurable attraction forces, represented in Figure 2. Each force represents a different sociological hypothesis regarding how scientific knowledge diffuses through a population, and their simultaneous execution generates the model's complex systems behavior.

- The **social force** assumes that scientists rely heavily on their professional networks to guide their research trajectories. An agent initiates a random walk across its topological graph, selecting a target node within a maximum of three degrees of separation. If the spatial distance to this social target's current epistemic position falls below the absorptive capacity threshold, the agent traverses toward it using the standard logistic velocity.

- The **semantic closeness force** assumes that scientists are influenced primarily by those working in similar paradigms, regardless of formal social ties. Under this logic, the agent ignores the social graph and scans the continuous spatial environment for the a random active agent within the defined distance threshold, adopting them as the primary focal point.
- The **gravity force** simulates scenarios where researchers migrate to highly populated, trendy topics. At the beginning of this specific routine, every grid node in the environment recalculates its local population variable by exhaustively querying the number of active agents within a set radius. The agent then evaluates all immediately neighboring grid nodes and adjusts its trajectory to face the neighbor exhibiting the absolute highest population density. The velocity calculation here is uniquely modified; the logistic function substitutes spatial distance with the pure population value of the target location, meaning agents accelerate toward densely packed paradigms.
- The **centroid coherence force** operates on the assumption that a scientific discipline is anchored by a core, highly coherent summary model. This force directs agents toward the absolute geometric center of their currently occupied topic. Agents identify the specific centroid agent that matches their lattice node's color classification and move directly toward it, simulating a pull toward theoretical purity.
- Finally, the **conference aggregation force** simulates the effects that academic conferences have in reaching a general consensus. A random subset of twenty agents is dynamically tagged with a boolean flag representing attendance. The system calculates the aggregate center of mass of this specific cohort. All tagged attendees then temporarily bypass their standard routines to traverse toward this collective focal point, artificially forcing mixing between potentially distant paradigms.

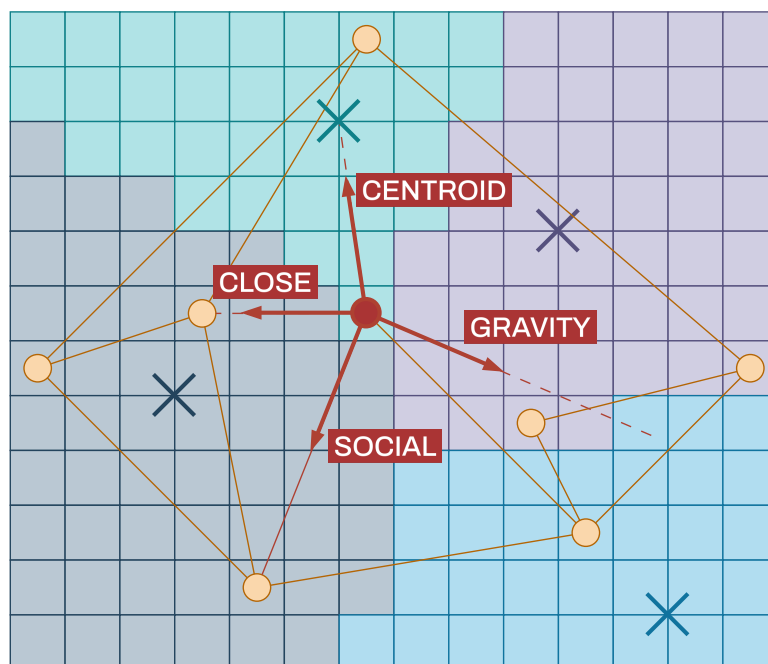


Figure 2: Forces that drive the movement of an agent through the epistemic landscape

Param	Description	Values
<code>dist</code>	Acts as a rigid threshold for absorptive capacity. It defines the maximum permissible spatial distance an agent can traverse toward a focal point in a single evaluation.	<code>[0.0, 100.0]</code>
<code>social-network</code>	Determines the initialization protocol for the social graph. The model can invoke file I/O to load historical nodes and edges, or can procedurally generate a scale-free network topology.	<code>{"data", "preferential attachment" }</code>
<code>social</code>	Toggles network-driven movement, activating the execution of the social and closeness logic calculations during the tick cycle.	<code>{ true, false }</code>
<code>gravity</code>	Toggles density-driven movement, activating the population recalculations across all spatial nodes.	<code>{ true, false }</code>
<code>centroid</code>	Toggles coherence-driven movement, directing agents toward the calculated geometric centers of their respective epistemic regions.	<code>{ true, false }</code>
<code>conferences</code>	Toggles event-driven aggregation, activating movement towards a center point for a randomly sampled subset of attendee agents.	<code>{ true, false }</code>
Ticks	The temporal duration of the simulation.	<code>(0, ∞)</code>

Table 5: Input parameters of the socio-epistemic knowledge spread model

2.2.4 Experimental input parameters and validation

Evaluating the performance of this model within an HPC framework requires a rigorous, systematic parameter sweep. The experimental space must push the simulation across varied phases of execution. The defined input variables for the model are detailed in Table 5.

The validation of the model will measure the *Epistemic Share* of each cluster under the same generated epistemic landscape in Netlogo and Repast HPC, to validate that the dynamics of both models are identical under a certain degree of confidence.

Chapter 3

Model implementations

The core objective of the upcoming third chapter of this Master’s thesis is the adaptation and implementation of the aforementioned social network models onto the Repast HPC platform. Transitioning the models from a single-threaded, high-level, interpreted environment like NetLogo into a highly distributed, compiled C++ and MPI-based framework requires profound architectural shifts and a deep understanding of parallel computing paradigms. The implementations must carefully map the conceptual socio-cognitive components of the ABM’s to Repast HPC’s strict distributed memory architecture.

3.1 Misinformation diffusion model

3.1.1 Parallelizing the model

Translating the extended SBFC model from its original NetLogo format into Repast HPC demands a fundamental shift in programming paradigms. While NetLogo relies on an interpreted, single-threaded execution loop that evaluates agents sequentially or in randomized batches, Repast HPC utilizes a compiled, distributed-memory C++ architecture built on top of the Message Passing Interface (MPI).

In a distributed environment, the global network of 4,039 Facebook nodes is not held in a single block of memory. Instead, the network graph and its constituent agents must be spatially partitioned across multiple processes (ranks). As the simulation scales to millions of agents in later benchmarking phases, the algorithms governing agent interaction, specifically the spreading functions evaluating neighbor states, must be carefully constructed to accommodate data that is non-local and potentially stale between synchronization steps.

Representing the misinformation diffusion agent in RepastHPC

In Repast HPC, the foundational memory container for the simulation is the `Context`. The `Context` encapsulates the agent population, ensuring through strict set semantics that only a single, unique instance of an agent exists within the logical bounds of the simulation. For this misinformation model, the `Context` will hold all instances of Scholars, Influencers, Normal agents, and Bots, managing their creation and lifecycle.

To represent the complex Facebook topology used in the original model simulations, the model must implement a `SharedNetwork` projection. A projection in Repast HPC is a construct that imposes a specific relational structure upon the otherwise unstructured

pool of agents residing in the **Context**. When an agent is instantiated and added to the **Context**, it automatically becomes a vertex in the **Network** projection, and the specific edges representing social connections are mapped according to the parsed dataset.

The temporal execution of the model is governed by Repast HPC’s dynamic discrete-event scheduler. Unlike simple loop-based simulators, the scheduler operates on a principle of conservative synchronization. At each tick, representing one hour of simulated time in the news cycle, the scheduler pops the next prioritized event from its queue. In the context of the misinformation model, a single scheduled tick involves a massive sequence of highly orchestrated operations for every active agent:

1. Query the current state (S, B, FC) and the specific class type (SC, I, N, BOT) of all connected neighbours via the **Network** projection.
2. Calculate the fractional ratios for the spreading functions $f_i(t)$ and $g_i(t)$.
3. Evaluate the independent verification (p_{verify}) and forgetting (p_{forget}) probabilities, dynamically accessing the specific parameters based on the agent’s assigned class.
4. Generate a high-entropy random number using the boost/MPI synchronized random number generators to trigger the stochastic transitions.
5. Determine and lock in the discrete state for $t + 1$.

Distributed memory management

Repast HPC is explicitly designed for distributed-memory hardware architectures, utilizing the Message Passing Interface (MPI) standard to parallelize operations across multiple separate compute nodes. In this architecture, the global **Context** and the massive **Network** projection do not exist as a single, contiguous entity in any single processor’s memory. Instead, the agents are spatially partitioned and distributed across multiple MPI processes, known as **ranks**.

Each MPI process is strictly and exclusively responsible for the computation of its assigned local agents. For example, if Process 1 (P1) holds Agent A, and Process 2 (P2) holds Agent B, P1 computes the transition logic for A, and P2 independently computes it for B. However, the highly interconnected nature of a scale-free social network guarantees that thousands of edges will span across process boundaries. If Agent A (residing on P1) is followers with Agent B (residing on P2), Agent A absolutely needs to read Agent B’s state to correctly calculate its local spreading functions $f_A(t)$ and $g_A(t)$.

Because physical memory is not shared in an MPI environment, P1 cannot simply dereference a pointer to read the memory address of Agent B on P2. Repast HPC resolves this fundamental distributed computing problem through the creation of “non-local” or “ghost” agents. In this system, P1 requests a copy of Agent B from P2 over the network. This ghost copy is instantiated and resides locally in P1’s **Context**. When Agent A iterates through its neighbours during the update step, it reads the local ghost copy of Agent B without needing to perform an expensive network call mid-computation.

Parallel communication management

Crucially, the code executing on P1 is strictly forbidden from altering or writing to the state of the ghost Agent B. Agent B is the sovereign property of its home process, P2. Because Agent B might stochastically change its state from Susceptible to Believer on P2 during a tick, the ghost copy sitting on P1 becomes instantly stale.

To maintain strict simulation coherence and prevent causality errors, Repast HPC requires the explicit, synchronized updating of all ghost agents across all process boundaries at the end of every tick. This complex data serialization is achieved using the Repast HPC **Package** pattern. The C++ implementation of the model must define:

- A **Package**: A lightweight, optimized C++ struct containing only the absolute minimal state data required for network transmission, in this case, the agent’s unique ID, its current belief state, and its class type.
- A **provider**: A routine on the home process (P2) that efficiently extracts the state from the real Agent B and serializes it into the **Package** for outbound MPI transmission.
- A **receiver**: A routine on the requesting process (P1) that receives the inbound **Package** array, deserializes the data, and maps the updates to the corresponding ghost Agent B.

The implementation will utilize Repast HPC’s default **GhostMode** synchronization protocol. In **GhostMode**, writes are strictly local, and distant agents are read from a ghost copy representing the stable system state at the previous time step ($t - 1$). While this ensures strict determinism and eliminates read-write race conditions, the mandatory block-synchronization of the entire boundary layer at the end of the tick induces massive latency, forming the primary barrier to scalability.

Figure 3 depicts how Repast HPC handles the distributed memory and parallel communication in the specific case of the heterogeneous SBFC model: The global network that holds the relationships between agents is partitioned to distribute the work load among the multiple processes or ranks. To read data from agents in foreign processes, the framework requests a ghost read-only copy to the provider process and hands it over to the receiver process.

3.1.2 Validating the Repast HPC implementation

In order to validate the correctness of the Repast HPC implementation of the misinformation diffusion model, a rigorous validation procedure must be established. The original NetLogo model’s behavior is well-documented, with specific emergent phenomena such as the formation of echo chambers, the critical thresholds for misinformation cascades, and the temporal dynamics of belief adoption. The Repast HPC implementation must be subjected to a battery of tests to ensure that it faithfully reproduces these key behaviors under identical initial conditions. This involves:

1. Running the Repast HPC model with the same initial network topology and agent class distributions as the original NetLogo model.

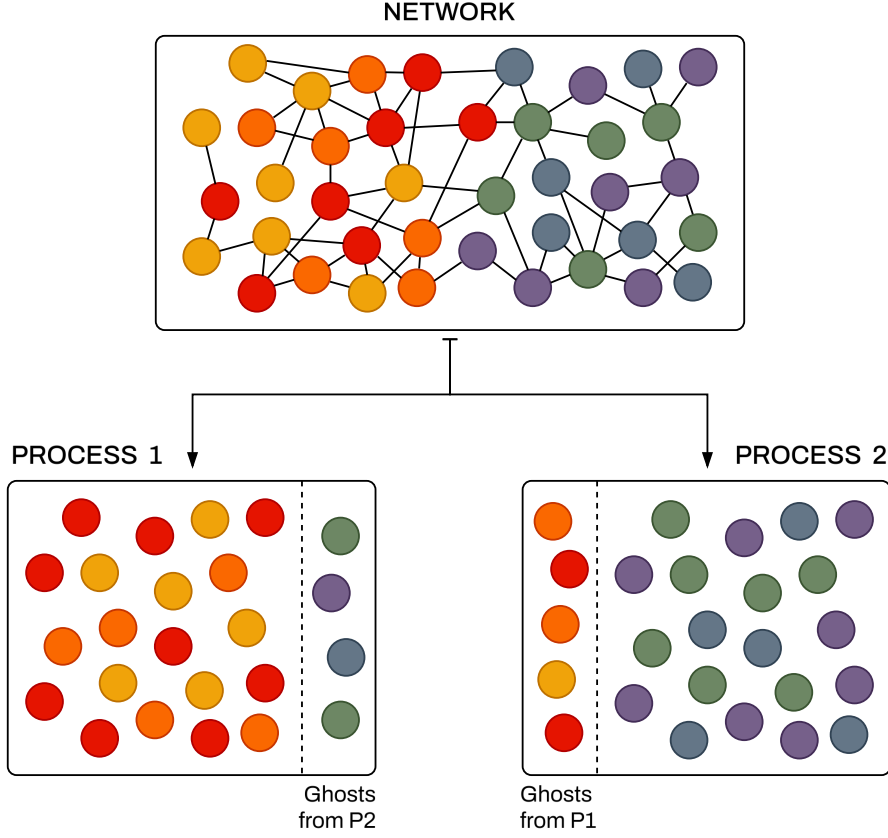


Figure 3: Repast HPC memory management applied to the SBFC model

2. Comparing the time series data of belief state distributions (S, B, FC) across both implementations to ensure they match within acceptable statistical variance.
3. Sampling across multiple random seeds to confirm that the stochastic dynamics are consistent and that the same emergent patterns arise in both implementations.
4. Performing sensitivity analyses on the key parameters (e.g., α hoax credibility, β spreading rate) to confirm that the Repast HPC model responds to parameter changes in the same manner as the NetLogo model.

To guarantee the integrity of the validation process, the same network topology is used in both scenarios, and a systematic sweep of the parameter space will be conducted, as outlined in Table 6. This table defines the specific parameter combinations that will be tested to ensure that the Repast HPC implementation captures the full range of behaviors exhibited by the original model.

Parameter Set	Samples	α	β	B_0	$p_{\text{verify_influencer}}$	$p_{\text{verify_scholar}}$
1	10	0.8	0.5	10	0.2	0.3
2	10	0.4	0.2	50	0.2	0.3
3	10	0.5	0.8	90	0.1	0.2

Table 6: Parameter sweeps for validating the Repast HPC implementation of the misinformation diffusion model

In Figure 4, Figure 5 and Figure 6, the results of the validation process are visualized. The time series plots compare the belief state distributions over time for both the

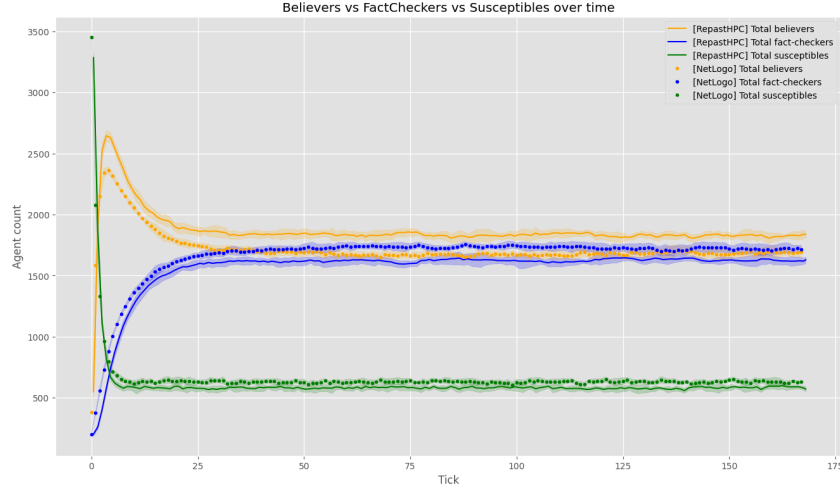


Figure 4: Validation of the Repast HPC implementation for parameter set 1

NetLogo and Repast HPC implementations under identical initial conditions, samples and parameter settings. The close alignment of the curves across multiple runs confirms that the Repast HPC implementation accurately reproduces the emergent dynamics of the original model, considering the standard deviation confidence interval.

The difference that can be observed in Figure 4, Figure 5 and Figure 6, plots can be attributed to the difference on the scholar community used for each simulation, as the original NetLogo model and the Repast HPC implementation used two different Fluid Communities algorithms to detect communities in the Facebook network, and the final results are not exactly the same. However, the overall dynamics of the belief state distributions are consistent across both implementations, confirming that the core mechanisms of misinformation diffusion are preserved in the Repast HPC version.

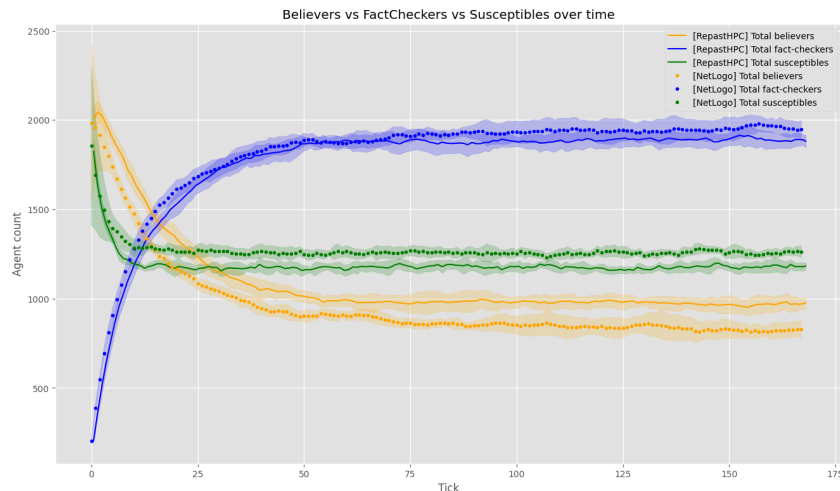


Figure 5: Validation of the Repast HPC implementation for parameter set 2

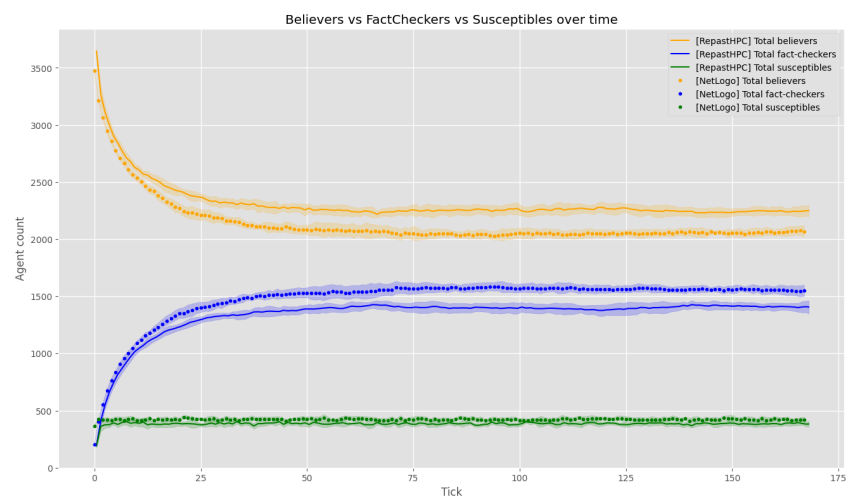


Figure 6: Validation of the Repast HPC implementation for parameter set 3

3.2 Socio-epistemic knowledge spread model

3.2.1 Parallelizing the model

Translating the socio-epistemic “*Opinion Dynamics of Science*” model from its native NetLogo environment to Repast HPC introduces architectural complexities fundamentally different from the misinformation diffusion model. While both rely on network topologies, the socio-epistemic model operates within a highly coupled dual-layer environment: agents traverse a continuous spatial lattice representing abstract mental models (the epistemic layer) while simultaneously evaluating connections across a multi-layered graph (the social layer).

Distributing this model across multiple MPI processes requires a careful orchestration of Repast HPC’s spatial partitioning, network projections, and ghost agent synchronization to ensure that the emergent formation of the giant component observed in the historical general relativity community is accurately and deterministically reproduced at scale.

Representing the socio-epistemic agent and dual projections

In Repast HPC, the global population of scientists is managed within a `repast::SharedContext`. To represent the complex environmental duality of the NetLogo model, this `Context` must be projected into two distinct spaces simultaneously:

- `repast::SharedContinuousSpace`: This projection replaces the discrete patches of NetLogo. It provides a continuous 2D Cartesian coordinate system where agents represent their current epistemic stance. The Euclidean distance between coordinates serves as the proxy for cognitive similarity.
- `repast::SharedNetwork`: Superimposed over the continuous space, this directed network projection manages the historical or procedural collaboration edges.

The local state variables that each individual scientist agent encapsulates are described in Table 7.

Spatial partitioning and the ghost buffer zone

The primary challenge in parallelizing this model lies in the spatial decomposition of the epistemic layer. Repast HPC divides the continuous space into geometric regions, assigning each region to a specific MPI rank. However, the model’s core attraction

Attribute	Description
<code>birth</code>	Serves as an age and rigidity modifier
<code>conference_flag</code>	Flag for temporary aggregate movement from the conference force
<code>topic_color</code>	Indicator for which epistemic cluster the agent belongs to

Table 7: Scientist agent internal attributes

forces, specifically the Semantic Closeness force and the Gravity force, require agents to evaluate the exact positions and densities of other agents within a specific radius.

Because an agent A near the boundary of node 1's spatial partition might need to evaluate an agent B residing on node 2, a *buffer zone* must be established. Repast HPC populates this buffer zone with ghost agents. The width of this buffer zone must be mathematically strictly greater than or equal to the maximum possible dist parameter configured for the simulation sweep. If the buffer is too narrow, the spatial density calculations driving the gravity force will experience boundary truncation errors, invalidating the simulation's emergent dynamics.

Synchronization and resolving the network-space conflict

The dual-layer nature of the model creates a severe parallelization conflict: two agents perfectly adjacent in the social network might reside on entirely opposite sides of the epistemic spatial grid, and therefore on completely different compute nodes. If agent A on node 1 needs to evaluate its co-author agent B on node 3 for the Social movement force, node 1 requires a network-based ghost of agent B, independent of the spatial buffer zone.

To maintain coherence across both projections, the Repast HPC simulation must execute a complex, two-tiered synchronization at the beginning of every tick. The implementation defines a `KnowledgeAgentPackage`: a lightweight C++ struct containing only the mutable properties of the agent.

During synchronization, the system first updates the status of all ghost agents across the network edges, ensuring topological consistency. Immediately following, it synchronizes the spatial buffer zones to ensure geographic consistency. Only after both synchronization barriers are cleared can the scheduler safely iterate over the local agents to calculate the logistic speed function and execute the movement algorithms.

3.2.2 Modifications to the original model

The Repast HPC implementation of the socio-epistemic model necessitated several modifications to the original NetLogo design to accommodate the constraints and capabilities of a distributed-memory architecture. The most significant changes include:

- **Removal of the conference force:** The original NetLogo model included a conference force that temporarily aggregated agents into a single point in the epistemic space during conference events. This mechanism was designed to simulate the intense intellectual exchange that occurs during academic conferences. However, implementing this force in a distributed environment proved to be infeasible due to the need for global synchronization and the potential for significant communication overhead. As a result, the conference force was removed from the Repast HPC implementation.
- **Modification of the speed function:** The original model used a logistic function to determine the speed of agent movement based on their birth and distance to the target point. In the Repast HPC implementation, this function was corrected

to ensure that the speed decreases with agent age and increases with distance, as originally intended. The corrected function is defined as:

$$\text{speed} = \frac{2}{1 + e^{-\left(\frac{\text{dist}}{\text{age}}\right)}} - 1 \quad (11)$$

- **Removal of the gravity force:** The gravity force in the original model caused agents to be attracted to areas of higher agent density in the epistemic space, simulating the tendency of scientists to cluster around popular research topics. However, this force required agents to evaluate the positions of all other agents within a certain radius, which is computationally expensive and difficult to manage in a distributed environment. Therefore, the gravity force was also removed from the Repast HPC implementation.
- **Removal of the spatial projection:** The synchronization of ghosts for both the network and the spatial projection was not possible due to lack of expertise in the Repast HPC framework and the positions were instead added as agent internal attributes.

3.2.3 Validating the Repast HPC implementation

To validate the correctness of the Repast HPC implementation of the socio-epistemic model, a comprehensive validation procedure was established. The original NetLogo model exhibited specific emergent phenomena, such as the clustering of agents in the epistemic space. The Repast HPC implementation was subjected to a series of tests to ensure that it faithfully reproduced these key behaviors under identical initial conditions. The validation process involved:

1. Running the Repast HPC model with the same initial network topology, agent distributions and epistemic space landscape as the original NetLogo model.
2. Comparing the time series data of the share of agents in each epistemic space cluster.
3. Sampling across multiple random seeds to confirm that the stochastic dynamics are consistent and that the same emergent patterns arise in both implementations.
4. Performing sensitivity analyses on the key parameters (e.g., influence of social forces, influence of centroid forces) to confirm that the Repast HPC model responds to parameter changes in the same manner as the NetLogo model.

The first step to guarantee a fair comparison between the two implementations is to use the same epistemic landscape in both scenarios. To do that, instead of leaving the cluster generation process to the randomness of each random number generator, the same cluster centers and standard deviations are defined in both implementations, ensuring that the initial conditions of the epistemic space are identical. Figure Figure 7 depicts the initial epistemic landscape used for the validation process, with the same cluster centers and standard deviations defined in both implementations.

The parameters used for the validation sweeps are defined in Table 8. This table outlines the specific parameter combinations that were tested to ensure that the Repast

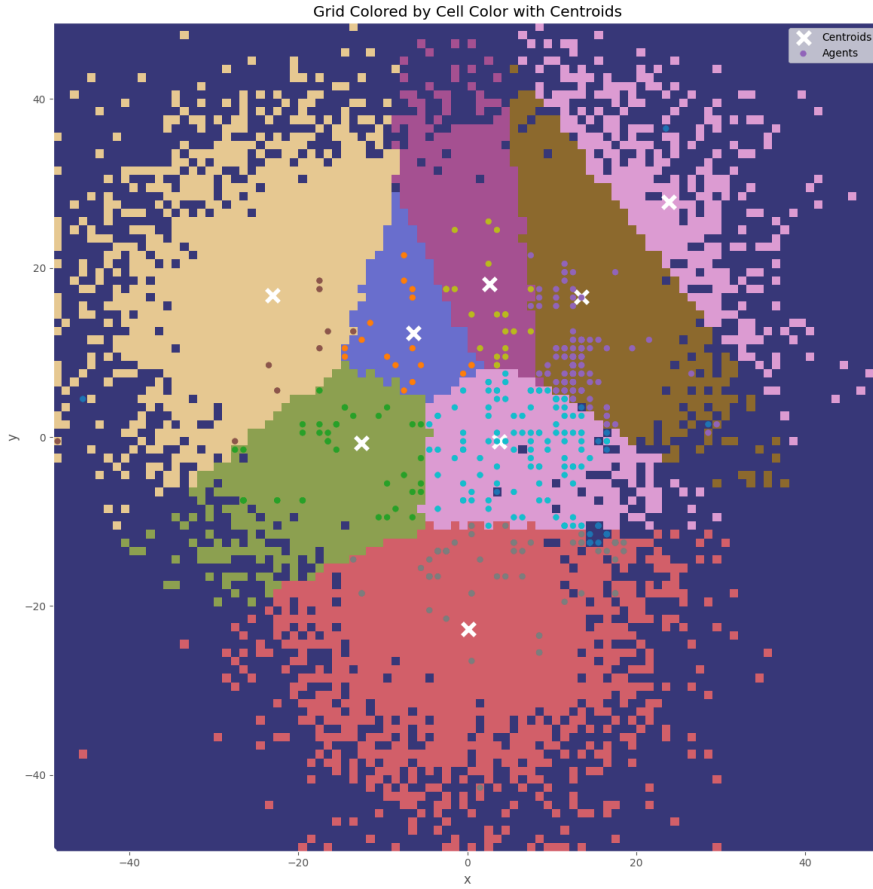


Figure 7: Initial epistemic landscape used for the validation process

HPC implementation captures the full range of behaviors exhibited by the original model.

Parameter Set	Samples	Social forces	Centroid forces
1	5	true	false
2	10	true	true
3	5	false	true

Table 8: Parameter sweeps for validating the Repast HPC implementation of the socio-epistemic model

In Figure 8, Figure 9 and Figure 10, the results of the validation process are visualized. The time series plots compare the share of agents in each epistemic cluster over time for both the NetLogo and Repast HPC implementations under identical initial conditions and parameter settings. The close alignment of the curves with the standard deviation from multiple runs confirms that the Repast HPC implementation accurately reproduces the emergent dynamics of the original model.

Also, the trajectory of the agents in the Repast HPC implementation is shown in Figure 11. The plot shows the movement of agents in the epistemic space over time, with different colors representing different clusters. The trajectories confirm that the agents

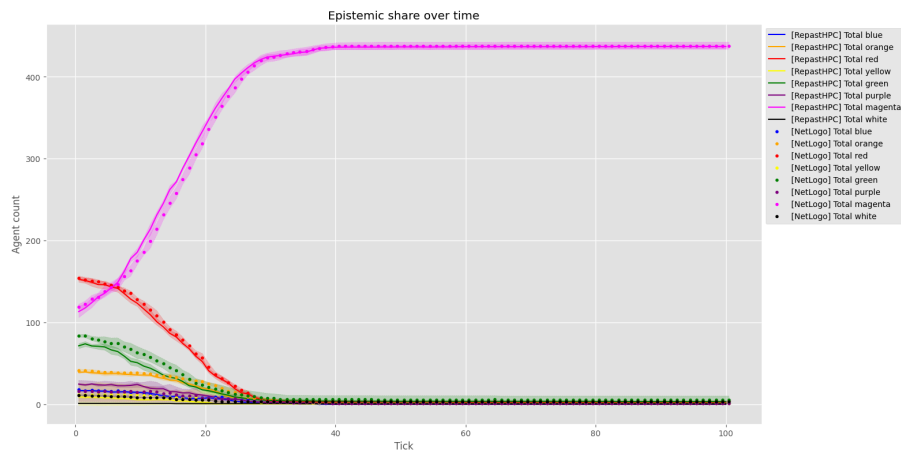


Figure 8: Validation of the Repast HPC implementation for parameter set 1

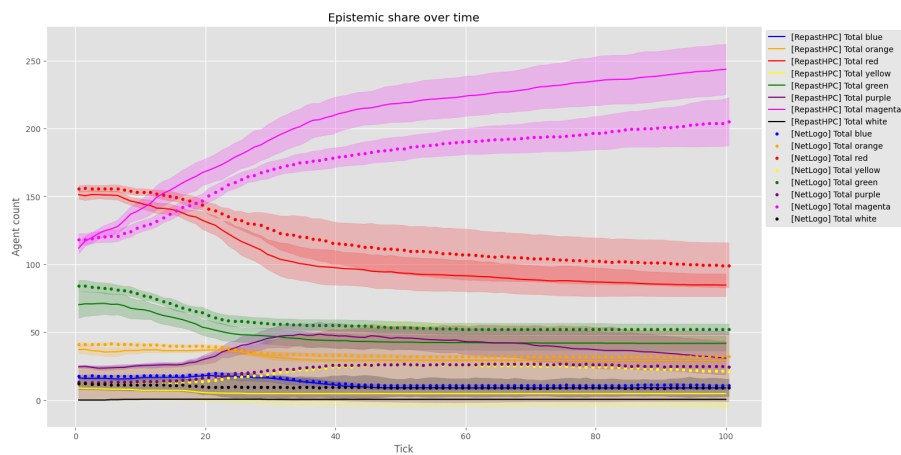


Figure 9: Validation of the Repast HPC implementation for parameter set 2

are clustering around the same epistemic regions, further validating the correctness of the Repast HPC version.

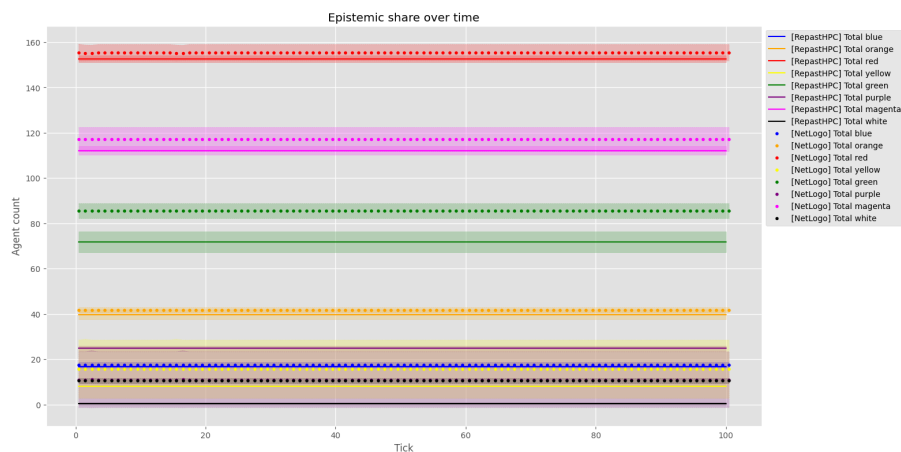


Figure 10: Validation of the Repast HPC implementation for parameter set 3

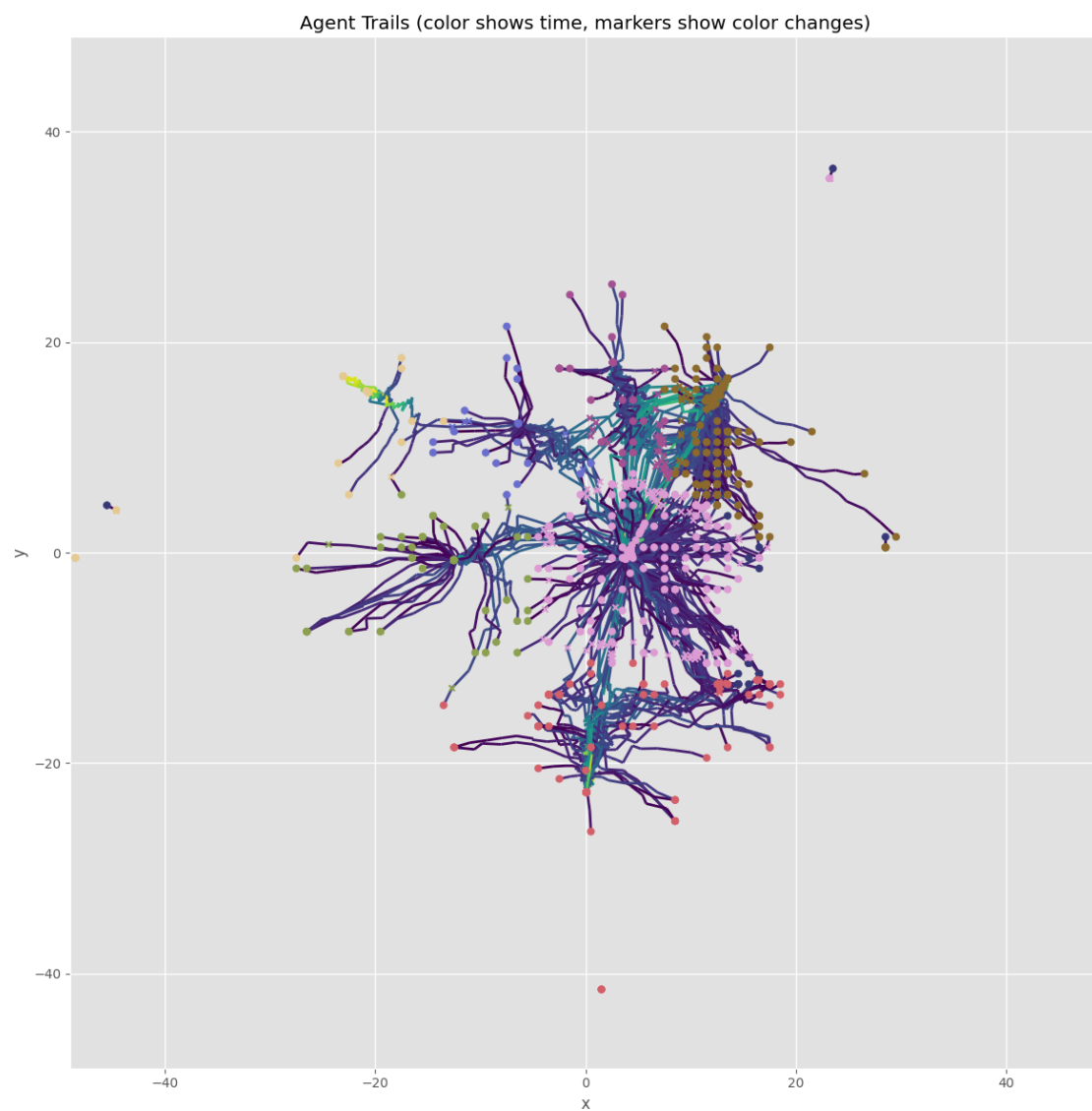


Figure 11: Trajectory of agents in the epistemic space over time in the Repast HPC implementation with parameter set 2

Chapter 4

Performance Analysis

This chapter presents the performance analysis of the implemented models. The models are evaluated based on their computational efficiency and scalability in simulating social network dynamics. The analysis presents the methodology and results for both the misinformation diffusion and the socio-epistemic knowledge spread models.

4.1 Methodology

Instrumentation for performance metering

In order to measure the performance of the RepastHPC model implementations, the TAU Performance System [22], [23] has been used. The Tuning and Analysis Utilities (TAU) system has been configured with the specific purpose of measuring the MPI message protocol performance, as well as execution time for the different testing scenarios. Python has been used to programatically analyze the results and create the figures.

To better understand the distribution of the execution time among the different model tasks, a set of specific timers have been set up. The description of each timer can be found in Table 9. This execution time breakdown is crucial to understand the performance bottlenecks of the model and to identify potential areas for optimization once the model parameters are explored.

Timer name	Description
Total time	Total execution time of the model.
Initialization time	Model initialization logic: network file parsing, network partitioning, agents instantiation, ghost agents initial synchronization.
Computation time	Model computation phase: calculation of next agent state and agent state updates.
Ghost Sync time	Model communication phase: inter-agent message exchange for ghost agents synchronization.
Tick Sync time	Time spent in synchronization calls at the end of each Repast Scheduler tick, specifically <code>MPI_Allreduce</code> method.
Results time	Model results collection and output phase: gathering of model results and writing to disk.

Table 9: Description of the timers used for performance analysis and execution time breakdown.

Execution time decomposition

The total execution time of the sequential implementation can be expressed as the sum of the initialization phase and the execution time of all simulation ticks:

$$T_{\text{total}} = T_{\text{init}} + \sum_{i=1}^N T_{\text{step},i} \quad (12)$$

where $N = 236$ corresponds to the number of independent ticks executed in each simulation.

For the RepastHPC implementation, the execution time includes a computation time and a synchronization overhead. The total execution time can therefore be decomposed as:

$$T_{\text{total}}^{\text{RHPC}} = T_{\text{init}}^{\text{RHPC}} + T_{\text{sync,init}} + \sum_{i=1}^N (T_{\text{step},i}^{\text{RHPC}} + T_{\text{sync},i}) \quad (13)$$

where $T_{\text{sync,init}}$ represents the synchronization time incurred during initialization and $T_{\text{sync},i}$ corresponds to the synchronization overhead at tick i .

The difference between the parallel and sequential execution times is then given by:

$$\Delta T_{\text{total}} = T_{\text{total}}^{\text{RHPC}} - T_{\text{total}} \quad (14)$$

which can be expanded as:

$$\Delta T_{\text{total}} = (T_{\text{init}}^{\text{RHPC}} - T_{\text{init}}) + T_{\text{sync,init}} + \sum_{i=1}^N ((T_{\text{step},i}^{\text{RHPC}} - T_{\text{step},i}) + T_{\text{sync},i}) \quad (15)$$

For the RepastHPC implementation to provide a performance improvement over the sequential execution, the condition

$$\Delta T_{\text{total}} < 0 \quad (16)$$

must hold. Consequently, the cumulative synchronization overhead must remain smaller than the computational savings achieved through parallelization:

$$T_{\text{sync,init}} + \sum_{i=1}^N T_{\text{sync},i} < (T_{\text{init}} - T_{\text{init}}^{\text{RHPC}}) + \sum_{i=1}^N (T_{\text{step},i} - T_{\text{step},i}^{\text{RHPC}}) \quad (17)$$

Synchronization time is defined as the sum of two communication components: the synchronization of ghost agents between MPI ranks and the global tick synchronization performed at the end of each tick. Furthermore, the execution time of an individual tick can be decomposed into computation, synchronization and result-processing phases.

The synchronization metric reported throughout this chapter corresponds to the arithmetic mean of the total synchronization time measured across all MPI ranks. This metric provides a convenient estimate of communication overhead but does not represent the true global synchronization cost. A more accurate measure would require identifying each synchronization window individually and summing the maximum synchronization time observed across all ranks for every synchronization event. Due to limitations of the available trace-analysis workflow, extracting synchronization windows

at this level of detail proved impractical. As a result, the reported synchronization times should be interpreted as a lower-bound estimate of the actual synchronization overhead.

Default setup

When executing the performance analysis, a systematic exploration of the model parameters has been conducted to understand their impact on the performance metrics. The parameters studied include the number of MPI ranks, the number of agents in the synthetic network, the type of synthetic network used, the average degree of the network, the partitioning strategy for distributing the network load across MPI ranks, and two synthetic factors: MPI message size and computational load.

Each model has a default setting for each of the parameters explored that has been chosen based on a balance between computational feasibility and representativeness of real-world scenarios. This default setup serves as a baseline for studying the impact of the mentioned parameters on the performance metrics.

Network topologies

For the network setup, a set of synthetic networks have been generated to evaluate the performance of the misinformation diffusion model. The synthetic network generation process allows the creation of controlled environments with specific characteristics, such as network size, degree distribution and clustering coefficient. Understanding the structural properties of the networks used in the performance analysis is essential for interpreting the observed performance results.

Three synthetic network models were selected to represent distinct structural scenarios commonly encountered in social network analysis. Their differing degree distributions and clustering properties make it possible to evaluate how network topology influences communication overhead, partition quality and scalability. Figure 12 illustrates the degree distribution of each generated networks.

1. **Holme-Kim** network [24]. This model generates scale-free networks with a power-law degree distribution while allowing control over the average clustering coefficient. Since both heavy-tailed degree distributions and high clustering are characteristic features of many real-world social networks, the Holme-Kim model serves as the primary benchmark topology throughout this study.
2. **Watts-Strogatz** network [25]. This model produces networks with a relatively narrow degree distribution and a tunable clustering coefficient. Due to its highly clustered structure and more homogeneous degree distribution, it provides a favourable scenario for graph partitioning algorithms.
3. **Barabási-Albert** network [26]. Similar to the Holme-Kim model, it generates scale-free networks with a power-law degree distribution. However, it exhibits significantly lower clustering coefficients. As a result, it represents a structural scenario that contrasts with the Watts-Strogatz model, allowing the evaluation of partitioning strategies under conditions where highly connected hubs dominate the network topology but local clustering is limited.

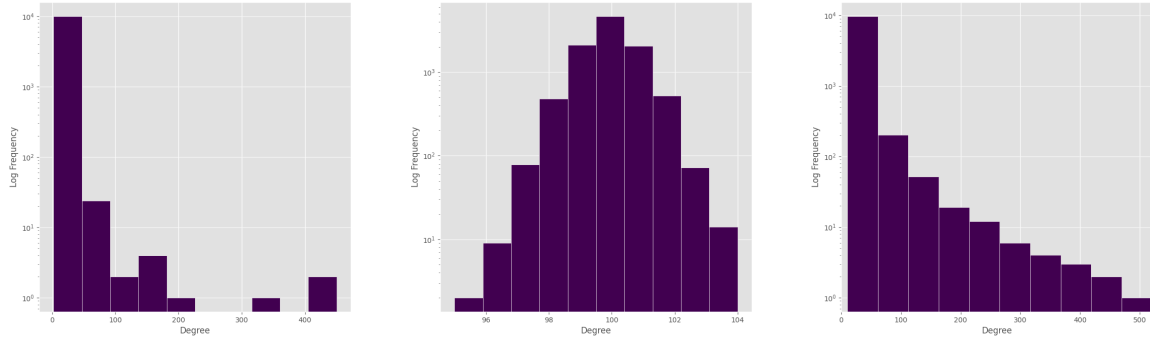


Figure 12: Degree distribution for, left to right, Holme-Kim, Watts-Strogatz and Barabási-Albert.

Each simulation run has been executed multiple times to account for variability in execution time and performance metrics. The results presented in the following sections are based on the average values obtained from these runs.

4.2 Misinformation diffusion model results

As mentioned in the methodology section, the parameters studied in the performance analysis have a default or baseline configuration. Table 10 provides a detailed description of the default configuration used for the performance analysis of the misinformation diffusion model.

Parameter	Description	Default value
Ranks	Number of MPI ranks used for the simulation.	4 ranks
Agents	Number of nodes in the synthetic network.	10^5 agents
Network	Type of synthetic network used for the simulation.	Holme and Kim
Avg degree	Average degree of the synthetic network.	4
Partition	Strategy used for partitioning the network across MPI ranks.	Fluid Communities
Msg extra size	Size of the messages exchanged between agents during the communication phase.	0 bytes
CPU extra load	Computational load of the agent state update function, measured in operations.	0 operations

Table 10: Default configuration for the performance analysis of the misinformation diffusion model.

Partitioning strategy analysis

The naive partitioning strategy assigns agents to MPI ranks according to a modulo operation on the agent identifier. As a result, the assignment is effectively independent of the network topology and can be regarded as a pseudo-random distribution of agents across ranks. In contrast, the Fluid Communities [27] partitioning strategy first identifies network communities and subsequently assigns communities to MPI ranks. Agents that belong to the same community are therefore placed within the same rank.

Figure 13 compares the execution time breakdown obtained using the naive and Fluid Communities partitioning strategies. The results show that the fluid communities strategy consistently outperforms the naive approach. The improvement is primarily explained by a reduction in synchronization time, while computation time remains largely unchanged. Since neighbouring agents tend to belong to the same community, a larger fraction of agent interactions occurs locally within each rank. Consequently, the number of ghost agents and the volume of inter-rank communication are reduced. Figure 14 compares the communication overhead of the two partitioning strategies. The volume of inbound and outbound communication for all agents is drastically reduced when partitioning the network based on the fluid communities algorithm.

These results demonstrate that partition quality is a critical factor in the performance of distributed agent-based simulations. By preserving network locality, community-aware partitioning reduces communication overhead and improves overall execution time.

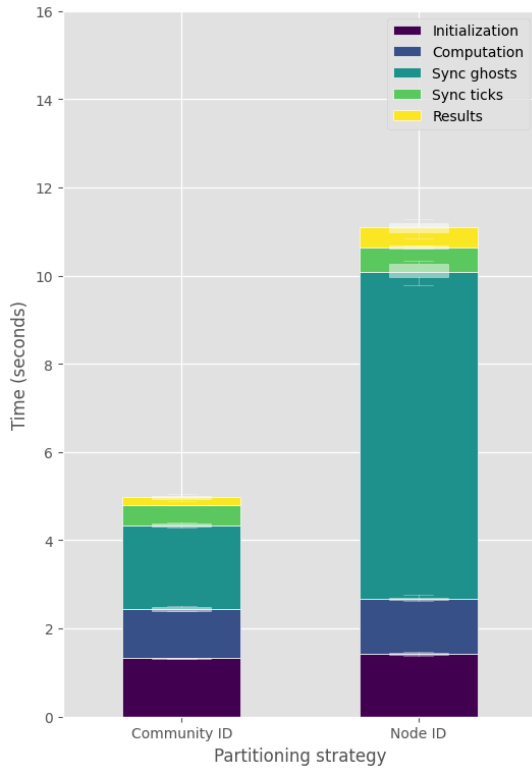


Figure 13: Execution time breakdown per partitioning strategy.

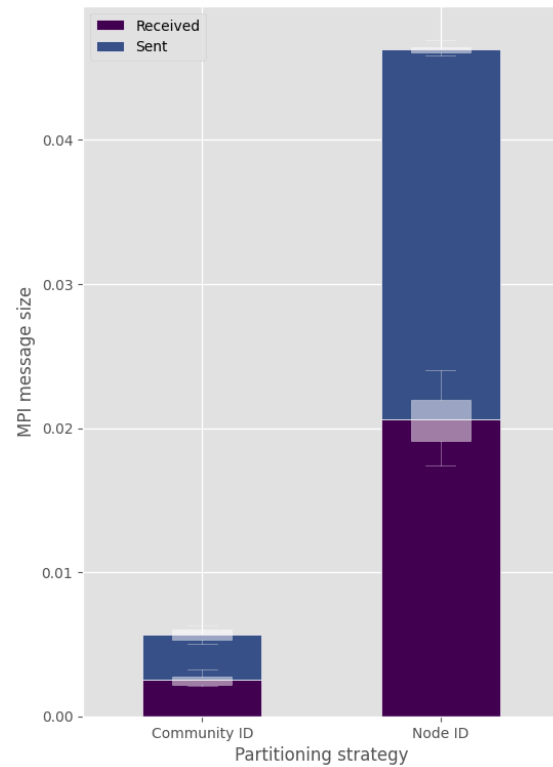


Figure 14: MPI message size by message direction per partition strategy.

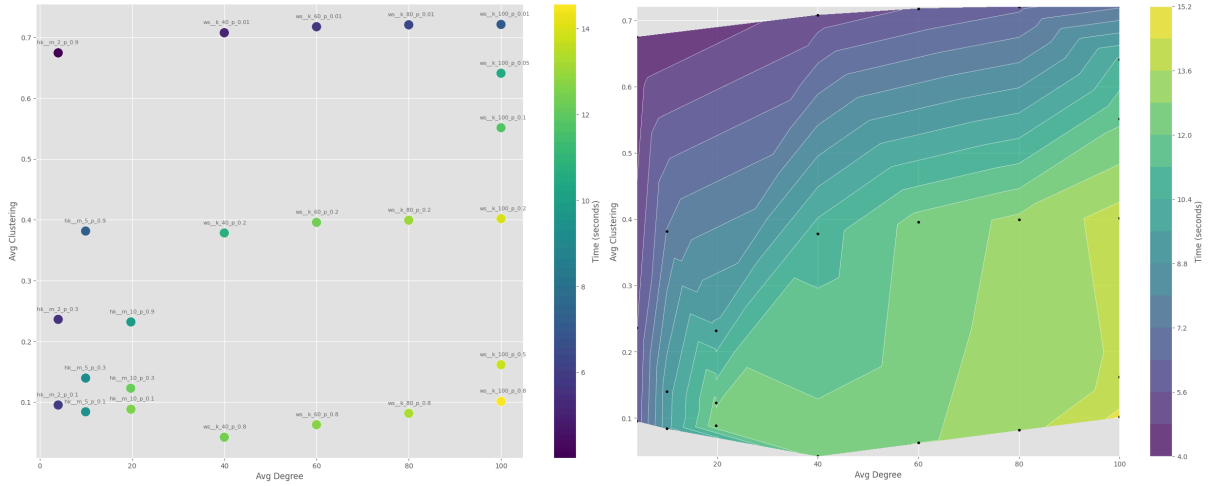


Figure 15: Total execution time for each network type, positioned by network average degree and average clustering coefficient. Interpolated results on the right.

Network structure analysis

Figure 15 presents the impact of network topology on the execution time breakdown. The analysis focuses on two structural properties: the average degree and the average clustering coefficient. The results indicate a strong dependency on the average clustering coefficient, whereas the influence of the average degree is comparatively weaker. As the clustering coefficient decreases, synchronization costs increase significantly and become a larger fraction of the total execution time. This behaviour can be explained by the characteristics of the fluid communities partitioning strategy. Highly clustered networks naturally contain well-defined communities, allowing connected agents to be grouped within the same rank. When the clustering coefficient decreases, community boundaries become less distinct and the partitioning algorithm is less effective at preserving locality. Consequently, a larger number of ghost agents must be maintained across rank boundaries, increasing communication overhead.

These findings suggest that network clustering is one of the most influential structural factors affecting distributed simulation performance, particularly when community-based partitioning strategies are employed.

Rank analysis

Figure 16 shows the effect of increasing the number of MPI ranks on the execution time breakdown for the different network topologies considered. The observed behaviour differs substantially between the network models. For the highly clustered network topology, total execution time decreases as the number of ranks increases, indicating that the computational savings obtained through parallelization outweigh the additional synchronization overhead. In contrast, for the weakly clustered scale-free network, execution time increases with the number of ranks.

The opposite trends can be explained by the effectiveness of the partitioning strategy. In networks with well-defined communities, partitioning successfully minimizes inter-rank communication, allowing the computational workload to be distributed efficiently.

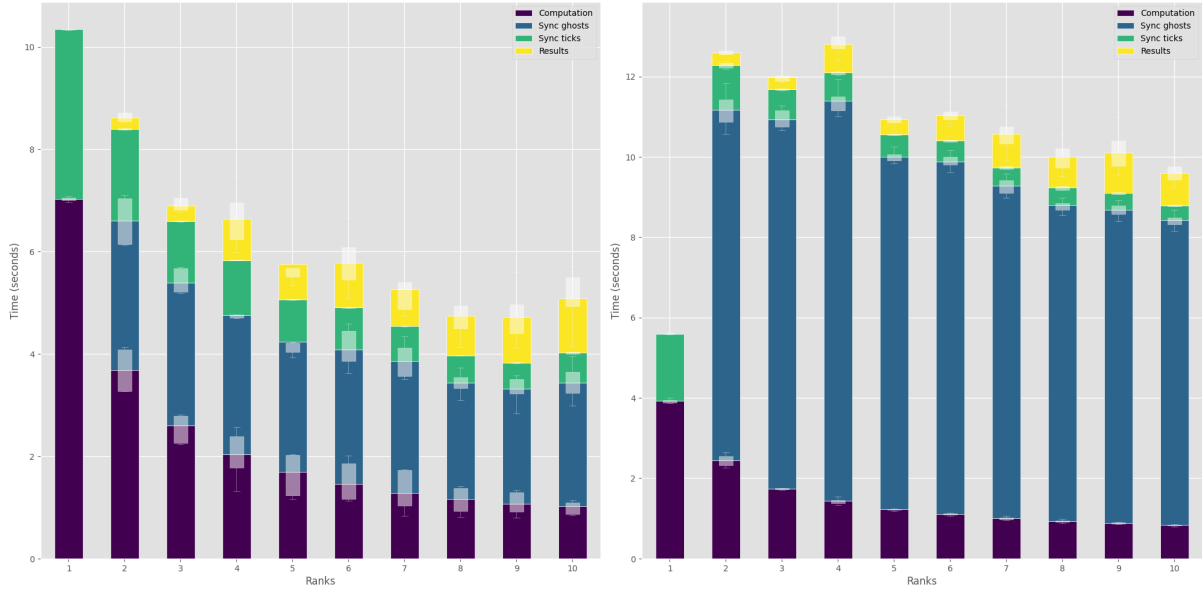


Figure 16: Execution time breakdown per number of total MPI ranks for Holme-Kim (left) and Barabási-Albert (right) networks.

Conversely, in networks where community structure is weak, the number of ghost agents grows rapidly as additional ranks are introduced. Under these conditions, synchronization costs increase faster than computation costs decrease.

The results highlight that increasing the number of MPI ranks does not necessarily improve performance. The effectiveness of parallelization is strongly dependent on the ability of the partitioning strategy to preserve network locality and limit communication overhead.

Agent count analysis

Figure 17 presents the effect of increasing the number of agents on the execution time breakdown. As expected, both computation and synchronization costs increase with the size of the network. The observed growth is approximately linear with respect to the number of agents, indicating that the computational workload and communication requirements scale proportionally to the problem size within the range explored.

The linear trend suggests that the implementation scales predictably with increasing network size. Although larger networks require additional communication due to the increased number of agent interactions, synchronization costs do not exhibit evidence of linear growth under the evaluated conditions.

Rank-Agent dependency

In Figure 18, the execution time breakdown for different combinations of ranks and agents is presented. The results indicate that the performance improvement from increasing the number of ranks is more pronounced with a larger number of agents, suggesting that the model benefits from parallelism as the computational load increases. Note how the Barabási-Albert results show a much less pronounced improvement as

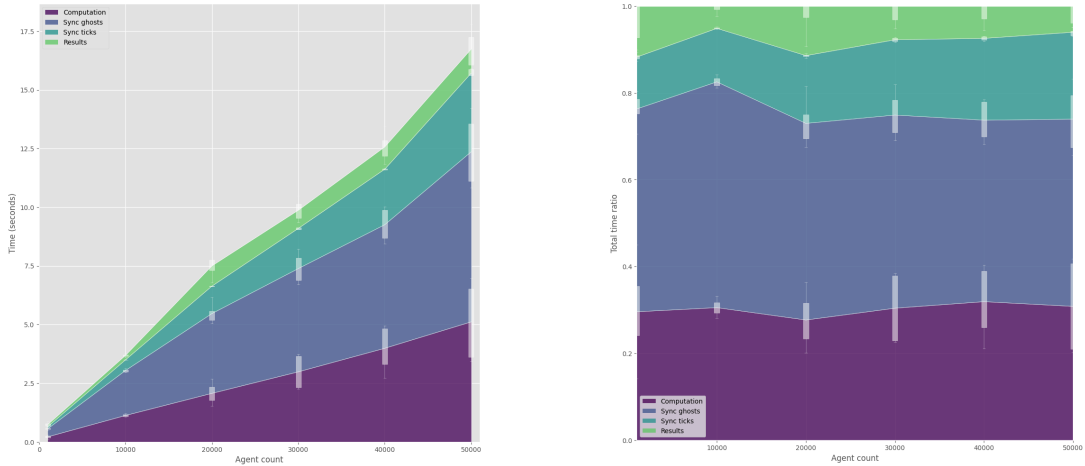


Figure 17: Execution time breakdown per number of agents in absolute (left) and relative (right) time.

the number of ranks and agents increase, and a dip in performance when changing from single-rank execution to parallel execution.

Computational load analysis

Figure 19 presents the effect of increasing the synthetic computational load applied during the agent update phase. In the context of the misinformation diffusion model, this parameter can be interpreted as representing increasingly complex decision-making processes when agents evaluate the credibility of information. The results show a linear increase in computation time as the additional workload increases. In contrast, synchronization times remain largely unaffected, since the communication pattern of the model does not change with the complexity of the agent update logic. As a consequence, the relative contribution of communication overhead decreases for computationally intensive scenarios. This behaviour is characteristic of distributed applications in which communication costs remain constant while the amount of local computation increases.

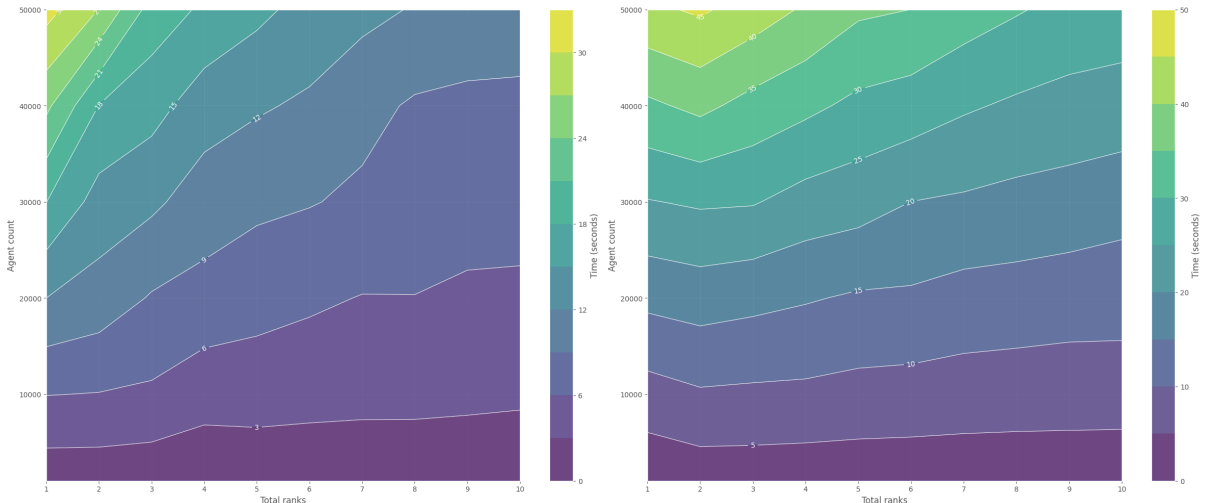


Figure 18: Execution time breakdown for different combinations of ranks and agents for Holme-Kim (left) and Barabási-Albert (right) networks.

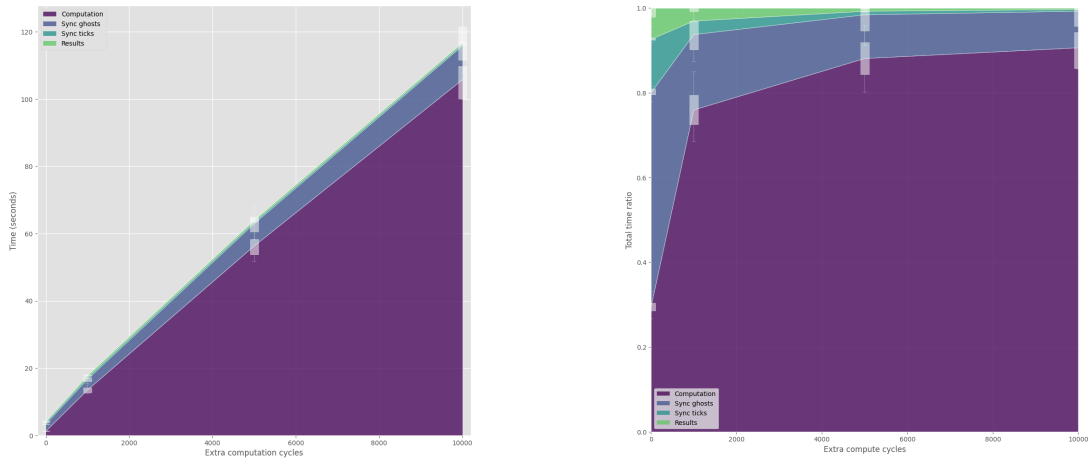


Figure 19: Execution time breakdown per extra computational load in absolute (left) and relative (right) time.

These results indicate that the parallel implementation becomes increasingly advantageous as the computational complexity of individual agent updates grows.

MPI Message size analysis

Figure 20 presents the effect of increasing the synthetic MPI message size exchanged during the ghosts synchronization phase. In the context of the misinformation diffusion model, this parameter can be interpreted as representing more information during the communication of the hoax and current belief status via messages between agents. The results show a linear increase in synchronization time as the additional message size increases. In contrast, computational times remain unaffected for the same reason presented in the computational load analysis.

These results indicate that the parallel implementation becomes increasingly disadvantageous as the individual agent communication package size grows.

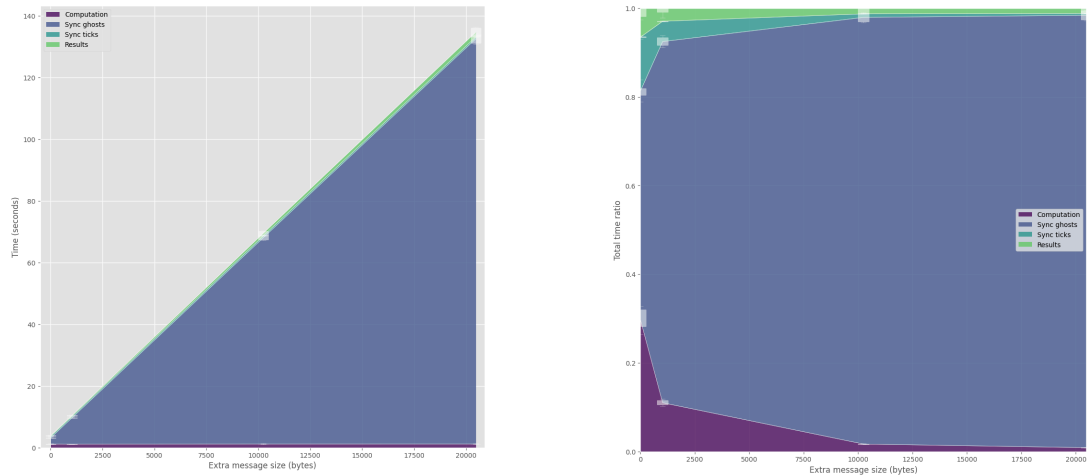


Figure 20: Execution time breakdown per extra MPI message size in absolute (left) and relative (right) time.

Parameter	Description	Default value
Ranks	Number of MPI ranks used for the simulation.	6 ranks
Agents	Number of nodes in the synthetic network.	10^3 agents
Network	Type of synthetic network used for the simulation.	Holme and Kim
Avg degree	Average degree of the synthetic network.	4
Partition	Strategy used for partitioning the network across MPI ranks.	Louvain
Msg extra size	Size of the messages exchanged between agents during the communication phase.	0 bytes
CPU extra load	Computational load of the agent state update function, measured in operations.	0 operations

Table 11: Default configuration for the performance analysis of the socio-epistemic knowledge spread model

4.3 Socio-epistemic knowledge spread model results

Table 11 provides a detailed description of the default configuration used for the performance analysis of the socio-epistemic knowledge spread model.

Methodological differences with the misinformation diffusion model

The performance analysis of the socio-epistemic knowledge spread model follows the same methodology used for the misinformation diffusion model. The same instrumentation strategy, execution-time breakdown and performance metrics were employed, allowing direct comparison between the two models.

A key difference lies in the network partitioning. In the misinformation diffusion model, community detection and partition assignment are performed during model initialization and it relies on the Fluid Communities [27] algorithm. In contrast, the socio-epistemic knowledge spread model performs partitioning during network generation, storing the resulting partition identifier as a node attribute within the network file. During simulation initialization, agents are directly assigned to MPI ranks according to this precomputed partition information. This approach reduces initialization overhead and ensures that repeated executions use identical network partitions. The downside is that the number of communities does not necessarily have to match to the total number of ranks, which can create load imbalances.

Another difference concerns the community detection algorithm employed. Whereas the misinformation diffusion model relies on the Fluid Communities algorithm, the socio-epistemic knowledge spread model uses the Louvain algorithm [28] to identify communities within the network.

Although the same timers are used in both models, some timers capture different operations. In particular, the initialization phase includes not only network loading and agent instantiation but also the generation of the epistemic landscape through a k-means clustering procedure. Furthermore, the ghost synchronization phase is more complex than in the misinformation diffusion model. In addition to synchronizing agent state, the communication phase must also support the temporal evolution of the network by activating edges whose creation time is reached during the simulation. Finally, The identification of ghost agents is also substantially more expensive. While the misinformation diffusion model only requires tracking immediate neighbours, the socio-epistemic knowledge spread model must maintain information about agents located up to three network hops away. Consequently, determining which ghost agents must be synchronized requires traversing a significantly larger neighbourhood around each local agent, increasing the cost associated with agent synchronization management.

Partitioning strategy analysis

Figure 21 compares the execution-time breakdown obtained using the Louvain-based partitioning strategy and the baseline Node-ID partitioning approach. The results in Figure 22 show that the Louvain partitioning strategy substantially reduces communication volume compared with the Node-ID strategy. This behaviour is expected, as community-aware partitioning places strongly connected agents within the same MPI rank and therefore reduces the number of ghost agents and inter-rank messages.

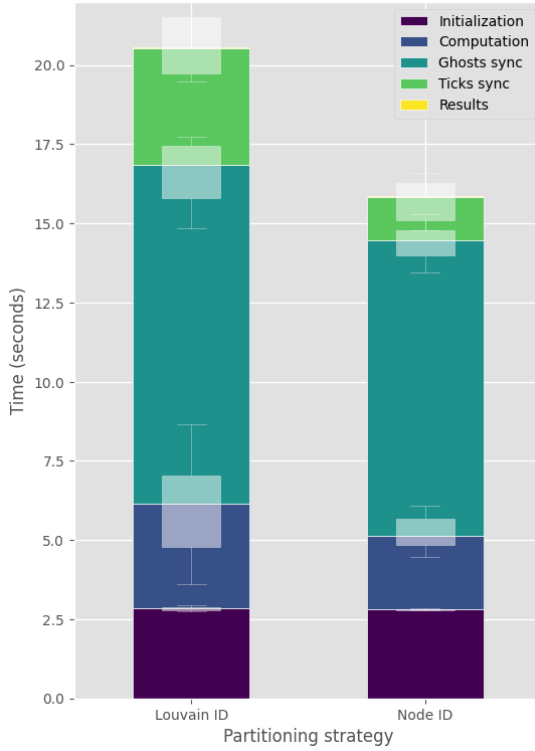


Figure 21: Execution time breakdown per partitioning strategy.

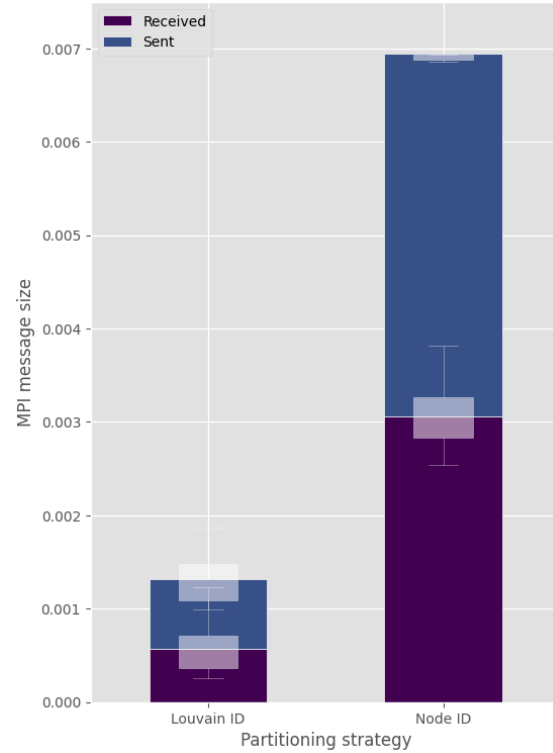


Figure 22: MPI message size by message direction per partition strategy.

However, unlike the misinformation diffusion model, the reduction in communication volume does not translate into a noticeable reduction in total execution time, but rather an increase. This behaviour suggests that communication itself is not the dominant bottleneck of the model. Instead, the cost associated with identifying and managing ghost agents appears to be more significant than the cost of exchanging synchronization messages. Trace analysis performed using Jumpshot supports this interpretation, revealing that a substantial fraction of the synchronization phase is spent determining which ghost agents require updates rather than performing MPI communication operations.

Network structure analysis

Figure 23 presents the impact of network topology on the execution-time breakdown. A notable difference emerges when comparing these results with those obtained for the misinformation diffusion model. Whereas the previous model exhibited a strong dependency on the average clustering coefficient, the socio-epistemic knowledge spread model is considerably more sensitive to the average degree of the network.

As the average degree increases, execution times grow substantially due to the larger number of neighbour relationships that must be maintained and synchronized across ranks. Since the model requires tracking information up to three hops away from each local agent, an increase in node degree leads to a rapid growth in the number of relevant neighbouring agents. Consequently, both ghost-management costs and synchronization overhead increase. By contrast, variations in the clustering coefficient have a smaller impact on performance. Although highly clustered networks may still improve partition quality, the benefit is reduced because the model requires information from agents that are multiple edges away. Under these conditions, preserving immediate community structure becomes less effective at reducing communication requirements.

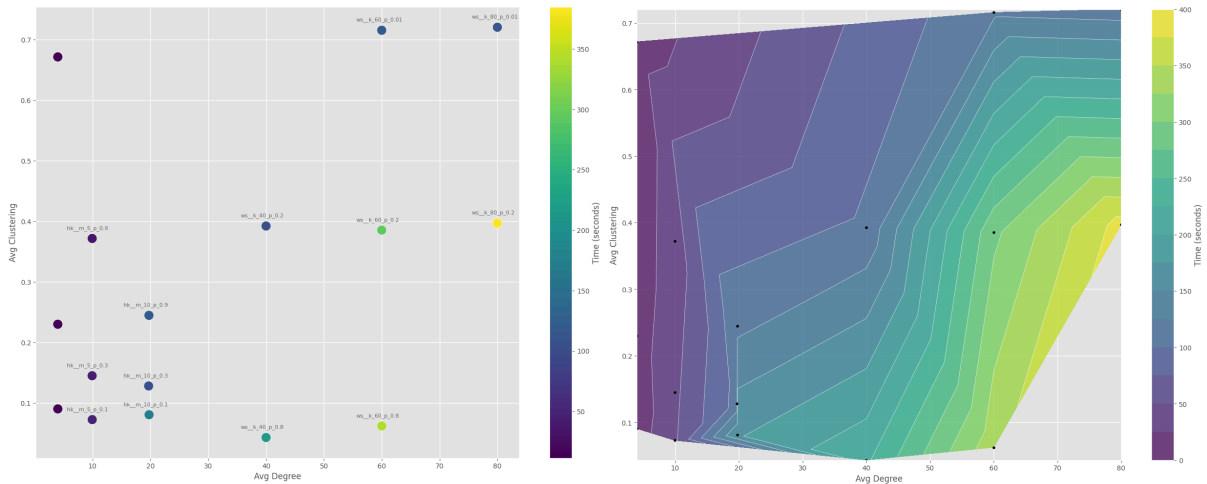


Figure 23: Total execution time for each network type, positioned by network average degree and average clustering coefficient. Interpolated results on the right.

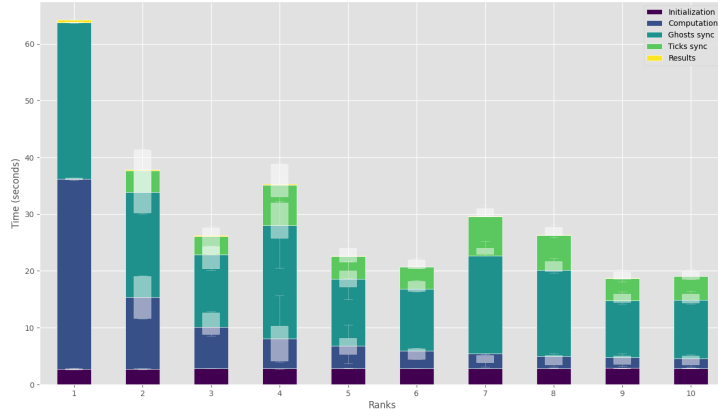


Figure 24: Execution time breakdown per number of total MPI ranks.

These findings suggest that the dominant performance bottleneck is not the existence of inter-rank connections themselves, but rather the number of neighbouring relationships that must be tracked and maintained throughout the simulation.

Rank analysis

Figure 24 shows the effect of increasing the number of MPI ranks on the execution-time breakdown. Compared with the misinformation diffusion model, a substantially larger fraction of the total execution time is spent in tick synchronization operations. This behaviour is attributed to load imbalance between MPI ranks.

The imbalance originates from the partitioning strategy employed. While the Louvain algorithm is effective at identifying community structure, it does not explicitly optimize for equal partition sizes. As a result, some MPI ranks may contain significantly more agents than others. Since global synchronization requires all ranks to progress at the same pace, faster ranks remain idle while waiting for slower ranks to complete their computations.

To further analyse the load imbalance between ranks, Figure 25 presents the distribution of MPI synchronization time used. The spread in samples represents the difference of time waiting for other ranks, thus showing the load imbalance is present for all total number of ranks except the single threaded execution.

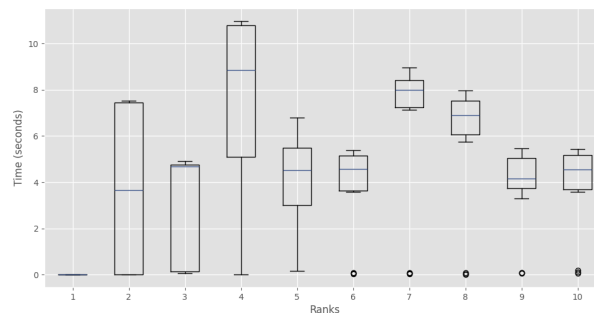


Figure 25: MPI Synchronization time per number of total MPI ranks.

Conclusions

This thesis has presented the implementation and performance analysis of two distributed Agent-Based Models (ABMs): a misinformation diffusion model and a socio-epistemic knowledge spread model. The work was motivated by the growing relevance of large-scale social simulations, the need to understand the factors that determine the performance and scalability of distributed ABM frameworks and the need to optimize compute power in the era of data.

The selected models were chosen according to a set of criteria centered around performance evaluation. In particular, both models include an explicit network structure defining static or dynamic relationships between agents and expose parameters that allow systematic exploration of computational and communication costs. These characteristics made them suitable case studies for identifying performance bottlenecks in distributed simulations.

A detailed analysis of the original model implementations was conducted, including the identification of the conceptual correspondence between the original NetLogo implementations and the Repast HPC framework. Based on this analysis, parallel versions of both models were successfully developed using Repast HPC. Validation experiments demonstrated that the distributed implementations reproduced the behaviour of the original models with sufficient accuracy, providing confidence that the performance results are representative of the intended simulations. One of the main technical challenges of the Repast HPC implementations was the management and synchronization of ghost agents, which constitute a fundamental mechanism for maintaining consistency across distributed network partitions. The development of the socio-epistemic knowledge spread model also highlighted some limitations of the available framework. While the network projection was successfully implemented, the spatial grid component of the original model proved significantly more challenging to reproduce within the scope of this work. Consequently, the spatial movement and force-based mechanisms of the original model were omitted from the distributed implementation, allowing the study to focus on the network interaction component.

The performance analysis was conducted through the instrumentation of the simulation code using the TAU Performance System. Execution traces, execution times and memory-related metrics were collected for a broad range of experimental configurations. Parameter sweeps were performed to evaluate the influence of network topology, partitioning strategy, problem size, computational load and communication requirements on the overall performance of the models.

The results demonstrate that partitioning quality is the dominant factor affecting the performance of distributed network-based agent simulations. Community-aware partitioning strategies significantly reduce ghost synchronization overhead by preserving network locality and minimizing inter-rank communication. In contrast, poorly partitioned networks may generate communication costs that exceed the computational

savings obtained through parallelization, leading to slower execution than the corresponding sequential implementation.

The effectiveness of the partitioning strategy was found to depend strongly on the structural characteristics of the underlying network. In particular, the average clustering coefficient exhibited a significant influence on performance, as highly clustered networks naturally form communities that can be partitioned efficiently. Networks with weaker community structures require a larger number of ghost agents and therefore incur greater synchronization overhead. By comparison, variations in network size, computational workload and message size produced results that were generally consistent with the expected scaling behaviour.

The analysis of the number of MPI ranks further highlighted the interaction between parallelization and communication overhead. Increasing the number of ranks only produced performance improvements when the reduction in computation time exceeded the additional synchronization costs. Consequently, the scalability of the distributed implementation depends not only on the available computational resources but also on the ability of the partitioning strategy to preserve locality within the network.

One observation that could not be fully explained was the increase in measured ghost synchronization time when additional computational workload was introduced. Although the communication pattern of the model remained unchanged, synchronization times exhibited a noticeable increase. Further investigation would be required to determine whether this behaviour originates from measurement methodology, MPI runtime effects, operating-system scheduling, or interactions between computation and communication phases.

A particularly relevant finding of this work is that the dominant performance bottleneck differs significantly between the two studied models, despite both being network-based distributed agent simulations. In the misinformation diffusion model, performance is primarily limited by communication volume. Consequently, reducing the number of ghost agents through effective community-based partitioning directly decreases synchronization overhead and leads to substantial performance improvements. In contrast, the socio-epistemic knowledge spread model is not primarily constrained by the cost of exchanging messages between MPI ranks, but rather by the computational effort required to identify, maintain and synchronize the extended neighbourhood information needed by the model. Since agents may require information from neighbours up to three connections away, the management of ghost agents becomes a significant source of overhead, often outweighing the communication costs themselves. This distinction highlights that scalability in distributed network-based ABMs depends not only on the amount of inter-process communication, but also on the complexity of the mechanisms required to support that communication.

Overall, the results confirm that distributed execution can significantly improve the performance of network-based agent simulations, emphasizing the importance of network topology and its partitioning strategy selection when developing scalable distributed ABMs.

Future Work

Extending the Repast HPC implementation to support spatial grid projections would allow a more faithful reproduction of models that combine network interactions with spatial dynamics. This extension would make it possible to evaluate the performance impact of simultaneously managing network and spatial projections in distributed environments.

Also, alternative partitioning strategies should be investigated. In particular, approaches that explicitly consider the total number of MPI ranks during partition construction may achieve better load balancing and lower communication overhead than the community-based strategy employed in this work.

The performance analysis could be extended beyond execution-time measurements by incorporating hardware performance counters through the Performance Application Programming Interface (PAPI). Such measurements would provide additional insight into cache behaviour, memory accesses and processor utilization, enabling a deeper understanding of the observed bottlenecks.

Finally, other performance-analysis tools, such as Scalasca [29], could be employed to obtain a finer-grained characterization of MPI communication patterns. In particular, these tools may facilitate a more accurate measurement of synchronization overhead and help explain some of the behaviours observed during the scalability experiments [30].

Acknowledgments

I would like to express my gratitude to my supervisors, Andreu Moreno and Josep Jorba, for their guidance, support and valuable feedback throughout the development of this thesis. Their expertise and encouragement have been fundamental in shaping this work and overcoming the challenges encountered during the research process.

Finally, I would like to thank everyone who supported me during the completion of my master's studies and this thesis.

Annex I: Source Code and Reproducibility

The complete source code for this Master Thesis work is available at the following URL:

<https://github.com/itsmgo/masters-thesis-abms-hpc>

It includes the Repast HPC implementations of both the misinformation diffusion model and the socio-epistemic knowledge spread model. The repository includes the necessary **Makefile** build configurations and scripts in the **tools** folder for running the simulations and reproducing the validation results presented in this work. The code is licensed under the MIT License, allowing for free use, modification, and distribution with proper attribution.

Bibliography

- [1] S. Coakley, M. Gheorghe, M. Holcombe, S. Chin, D. Worth, and C. Greenough, "Exploitation of high performance computing in the FLAME agent-based simulation framework," 2012, pp. 538–545. doi: 10.1109/HPCC.2012.79.
- [2] P. Richmond, S. Coakley, and D. M. Romano, "A high performance agent based modelling framework on graphics card hardware with CUDA," in *Proceedings of The 8th International Conference on Autonomous Agents and Multiagent Systems- Volume 2*, 2009, pp. 1125–1126.
- [3] N. T. Collier and M. J. North, "Parallel Agent-Based Simulation with Repast for High Performance Computing," *Simulation*, vol. 89, no. 10, pp. 1215–1235, 2013, doi: 10.1177/0037549712462620.
- [4] Repast Development Team, "RepastHPC: A High-Performance Agent-Based Modeling Platform." [Online]. Available: <https://repast.github.io/>
- [5] G. Cordasco, R. D. Chiara, A. Mancuso, D. Mazzeo, V. Scarano, and C. Spagnuolo, "Bringing together efficiency and effectiveness in distributed simulations: the experience with D-MASON," *Simulation*, vol. 89, no. 10, pp. 1236–1253, 2013, doi: 10.1177/0037549713493078.
- [6] A. Rousset, B. Herrmann, C. Lang, and L. Philippe, "A survey on parallel and distributed multi-agent systems for high performance computing simulations," *Computer Science Review*, vol. 22, pp. 27–46, 2016, doi: 10.1016/j.cosrev.2016.08.001.
- [7] A. Moreno, J. Jorba, C. Peralta, E. César, A. Sikora, and M. Hanzich, "A methodology for selecting a performance-convenient ABMS development framework on HPC platforms," *Simulation Modelling Practice and Theory*, 2023, doi: 10.1016/j.simpat.2023.102812.
- [8] N. D. Rollins, C. M. Barton, S. Bergin, M. A. Janssen, and A. Lee, "A Computational Model Library for Publishing Model Documentation and Code," *Environmental Modelling & Software*, vol. 61, pp. 59–64, 2014, doi: 10.1016/j.envsoft.2014.06.022.
- [9] CoMSES Net Community, "CoMSES Net Computational Model Library." [Online]. Available: <https://www.comses.net/codebases/>
- [10] A. Moreno, J. J. Rodriguez, D. Beltran, A. Sikora, J. Jorba, and E. Cesar, "Designing a benchmark for the performance evaluation of agent-based simulation applications on HPC," *The Journal of Supercomputing*, vol. 75, pp. 1524–1550, 2019, doi: 10.1007/s11227-018-2688-8.
- [11] A. K. Yalabadi, M. Yazdani-Jahromi, S. Abdizaji, I. Garibay, and O. O. Garibay, "Controlling the Misinformation Diffusion in Social Media by the Effect of Different Classes of Agents," 2024, doi: 10.48550/arXiv.2401.11524.
- [12] A. K. Yalabadi, "Controlling the misinformation diffusion in social media by the effect of different classes of agents, Version 1.0.0." [Online]. Available: <https://www.comses.net/codebases/48df051c-1f3a-4a27-94d0-bf2253947816/releases/1.0.0/>
- [13] M. Tambuscio, G. Ruffo, A. Flammini, and F. Menczer, "Fact-checking Effect on Viral Hoaxes: A Model of Misinformation Spread in Social Networks," in *Proceedings of the 24th International Conference on World Wide Web Companion (WWW '15 Companion)*, New York, NY, USA: ACM, 2015, pp. 977–982. doi: 10.1145/2740908.2742572.
- [14] E. Sulis and M. Tambuscio, "Simulation of misinformation spreading processes in social networks: an application with NetLogo," in *2020 IEEE 7th International Conference on Data Science and Advanced Analytics (DSAA)*, 2020, pp. 614–618. doi: 10.1109/DSAA49011.2020.00086.
- [15] A. Willmore, "This Analysis Shows How Viral Fake Election News Stories Outperformed Real News on Facebook." [Online]. Available: <https://www.buzzfeednews.com/article/craigsilverman/viral-fake-election-news-outperformed-real-news-on-facebook>
- [16] B. Buarque, "An Opinion Dynamics of Science? Agent-Based Modeling of Knowledge Spread." [Online]. Available: <https://www.comses.net/codebases/6896ae3b-4097-40fd-9a0a-c2d93085e346/releases/1.0.0/>
- [17] ModelSEN Project, "ModelSEN: Modelling Historical Knowledge Processes and the Evolution of Science." Accessed: July 29, 2024. [Online]. Available: <https://www.modelsen.org/>

- [18] R. Lalli, R. Howey, and D. Wintergrün, “The dynamics of collaboration networks and the history of general relativity, 1925–1970,” *Scientometrics*, 2019, doi: 10.1007/s11192-019-03327-1.
- [19] A. S. Blum, R. Lalli, and S. Walter, Eds., *The Renaissance of General Relativity in Context*. in Einstein Studies. Birkhäuser Basel, 2020. doi: 10.1007/978-1-4939-9745-9.
- [20] R. Schlattmann and M. Vogl, “Trajectories of Change: Approaches for Tracking Knowledge Evolution.” 2024. doi: 10.48550/arxiv.2501.00391.
- [21] R. Lalli, R. Howey, and D. Wintergrün, “The Socio-Epistemic Networks of General Relativity, 1925–1970,” 2020, pp. 15–84. doi: 10.1007/978-3-030-50754-1_2.
- [22] S. S. Shende and A. D. Malony, “The TAU Performance System,” in *Proceedings of the 2006 IEEE International Conference on Cluster Computing*, IEEE, 2006, pp. 1–14. doi: 10.1109/CLUSTER.2006.311839.
- [23] University of Oregon Performance Research Lab, “The TAU Performance System.”
- [24] P. Holme and B. J. Kim, “Growing scale-free networks with tunable clustering,” *Physical Review E*, 2002, doi: 10.1103/PhysRevE.65.026107.
- [25] D. J. Watts and S. H. Strogatz, “Collective dynamics of 'small-world' networks,” *Nature*, vol. 393, no. 6684, pp. 440–442, 1998, doi: 10.1038/30918.
- [26] A.-L. Barabasi and R. Albert, “Emergence of Scaling in Random Networks,” *Science*, vol. 286, no. 5439, pp. 509–512, 1999, doi: 10.1126/science.286.5439.509.
- [27] F. Parés *et al.*, “Fluid Communities: A Competitive, Scalable and Diverse Community Detection Algorithm,” in *Complex Networks & Their Applications VI: Proceedings of the Sixth International Conference on Complex Networks and Their Applications (COMPLEX NETWORKS 2017)*, 2017, pp. 229–240.
- [28] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, “Fast unfolding of communities in large networks,” *Journal of Statistical Mechanics: Theory and Experiment*, vol. 2008, no. 10, p. P10008, 2008.
- [29] Julich Supercomputing Centre, “Scalasca Trace Tools: Toolset for scalable performance analysis of large-scale parallel applications.” 2023. doi: 10.5281/zenodo.7440854.
- [30] Julich Supercomputing Centre, “Scalasca Performance Properties.” 2012. Accessed: June 11, 2026. [Online]. Available: https://apps.fz-juelich.de/scalasca/releases/scalasca/1.4/help/scalasca_patterns-1.4.html